

Ramaiah Institute of Technology

Department of AI & DS

Data Communication & Networking

Unit 5

Unit 5

Congestion control - Congestion, Congestion control, congestion control in TCP.

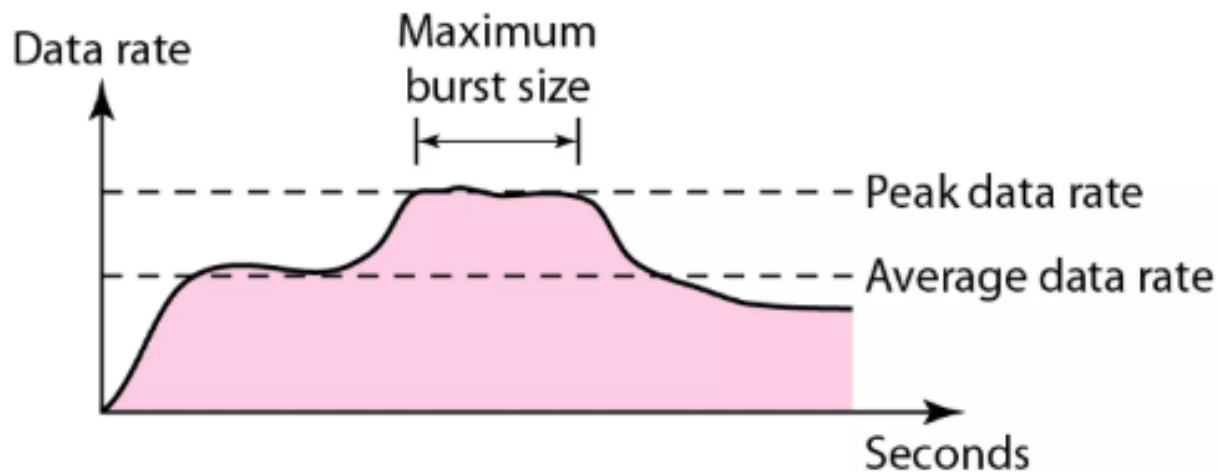
Application Layer: Domain Name System - Namespace, Domain name space, Distribution of Name space, DNS in internet, Resolution; Remote logging – TELNET; Electronic mail – Architecture, User Agent, Message Transfer Agent: SMTP; File transfer - File transfer protocol (FTP);

DATA TRAFFIC

Traffic Descriptor

- Traffic descriptors are qualitative values that represent a data flow

Figure 24.1 *Traffic descriptors*



DATA TRAFFIC

Average Data Rate

- The average data rate is the number of bits sent during a period of time, divided by the number of seconds in that period. We use the following equation:

$$\text{Average data rate} = \frac{\text{amount of data}}{\text{time}}$$

Peak Data Rate

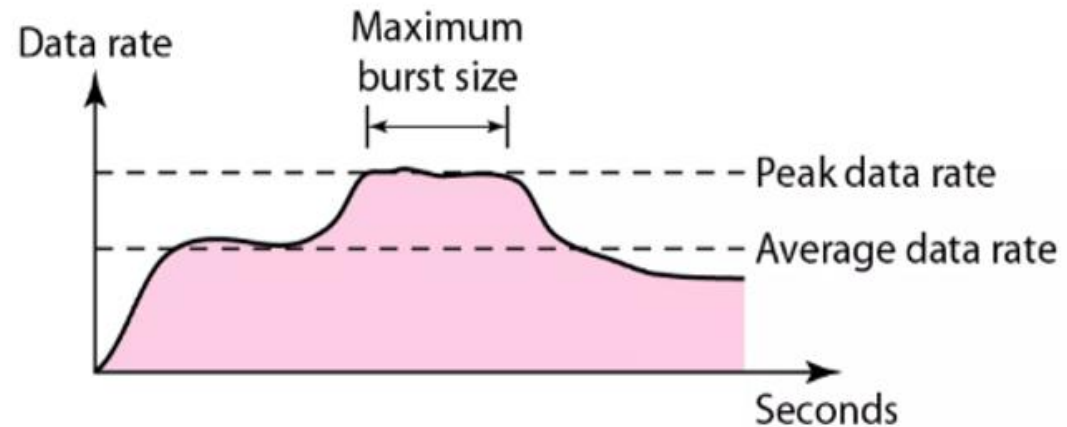
- The peak data rate defines the maximum data rate of the traffic. It is the maximum y axis value.

Maximum Burst Size

- The maximum burst size normally refers to the maximum length of time the traffic is generated at the peak rate.

Effective Bandwidth

- The effective bandwidth is a function of three values: average data rate, peak data rate, and maximum burst size.



DATA TRAFFIC

Traffic Profiles

For a data flow can have one of the following traffic profiles: constant bit rate, variable bit rate, or bursty .

Constant Bit Rate

- A constant-bit-rate (CBR), or a fixed-rate, traffic model has a **data rate that does not change**. In this type of flow, the average data rate and the peak data rate are the same. The maximum burst size is not applicable.

Variable Bit Rate

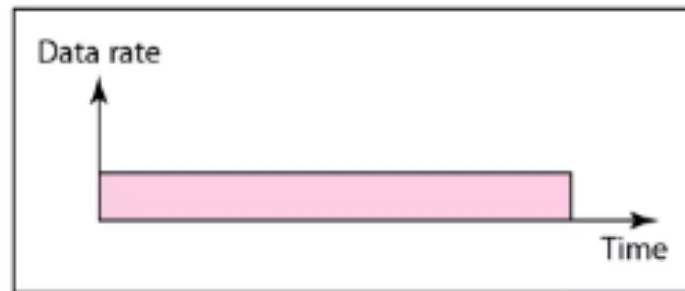
- In the variable-bit-rate (VBR) category, the rate of the data flow changes in time, with the **changes smooth instead of sudden and sharp**.

Bursty

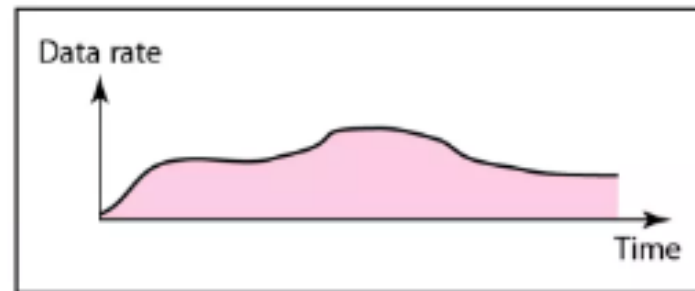
- In the bursty data category, the data rate changes suddenly in a very short time. It may jump from zero, for example, to 1 Mbps in a few microseconds and vice versa. It may also remain at this value for a while.

DATA TRAFFIC

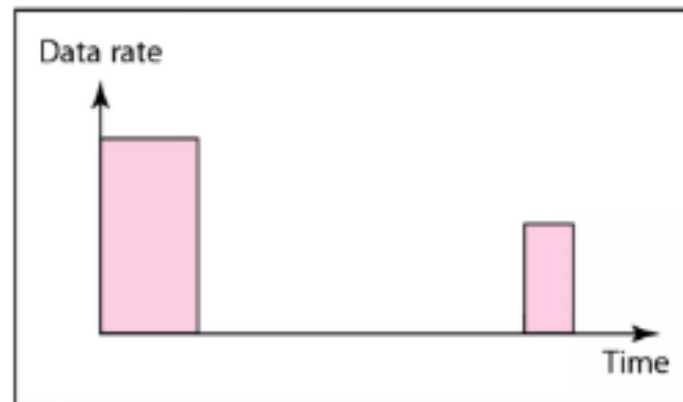
Figure 24.2 *Three traffic profiles*



a. Constant bit rate



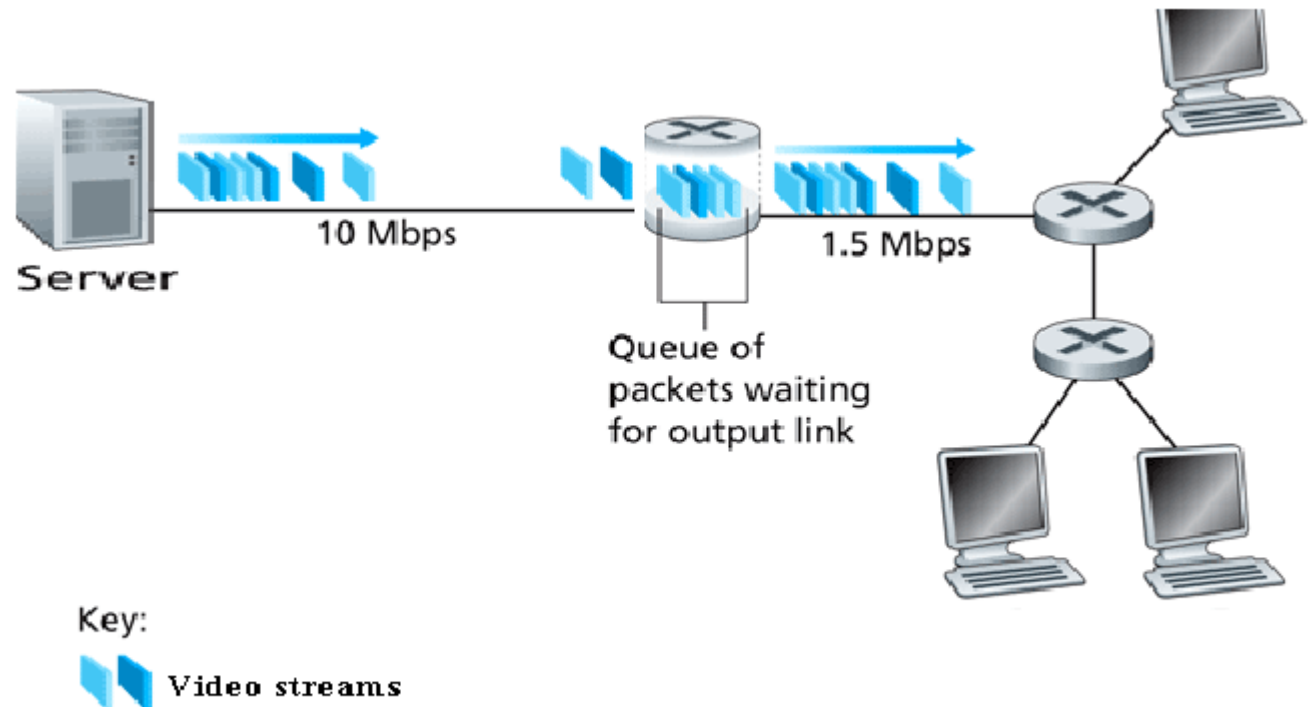
b. Variable bit rate



c. Bursty

CONGESTION

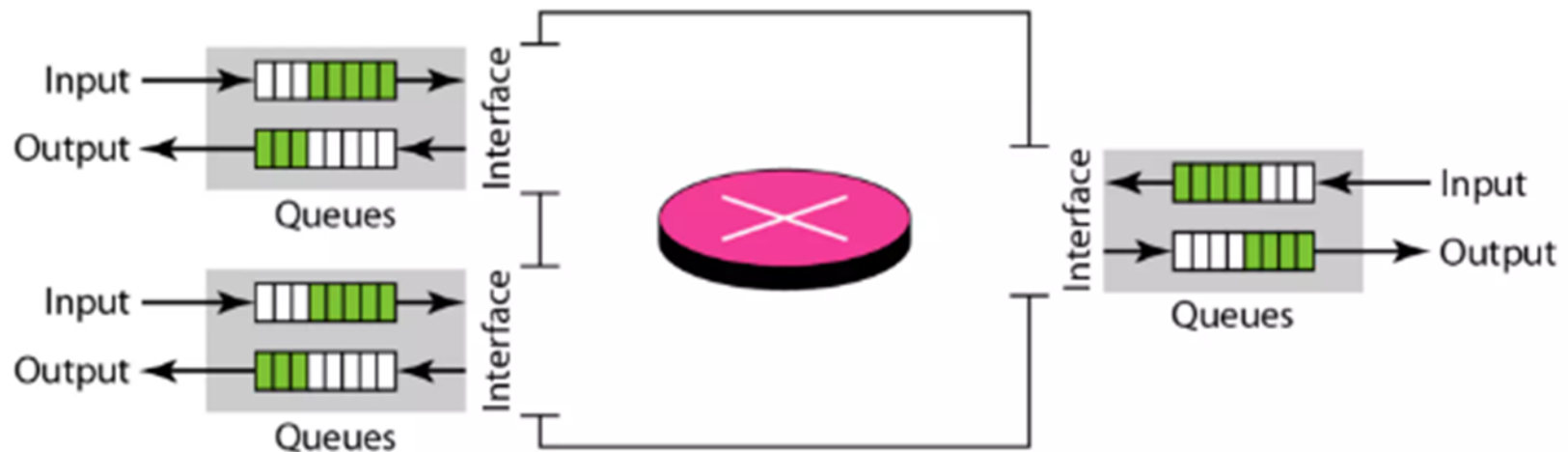
- Congestion in a network may occur if the **load on the network**-the number of packets sent to the network-is greater than the capacity of the network-the number of packets a network can handle.
- Congestion in a network or internetwork occurs because routers and switches have queues-buffers that hold the packets before and after processing.



CONGESTION

- Congestion control refers to the **mechanisms and techniques to control the congestion and keep the load below the capacity.**
- A router, for example, has an input queue and an output queue for each interface. When a packet arrives at the incoming interface, it undergoes three steps before departing, as shown in Figure 24.3.

Figure 24.3 *Queues in a router*



CONGESTION

Steps followed at queues in every router:

1. The packet is put at the end of the input queue while waiting to be checked.
2. The **processing module** of the router removes the packet from the input queue once it reaches the front of the queue and uses its **routing table** and the destination address to find the route.
3. The packet is put in the appropriate **output queue** and waits its turn to be sent.

We need to be aware of two issues.

1. If the **rate of packet arrival is higher than the packet processing rate**, the input queues become longer and longer.
2. If the **packet departure rate is less than the packet processing rate**, the output queues become longer and longer.

CONGESTION

Network Performance

Congestion control involves two factors that measure the performance of a network: delay and throughput.

Delay Versus Load

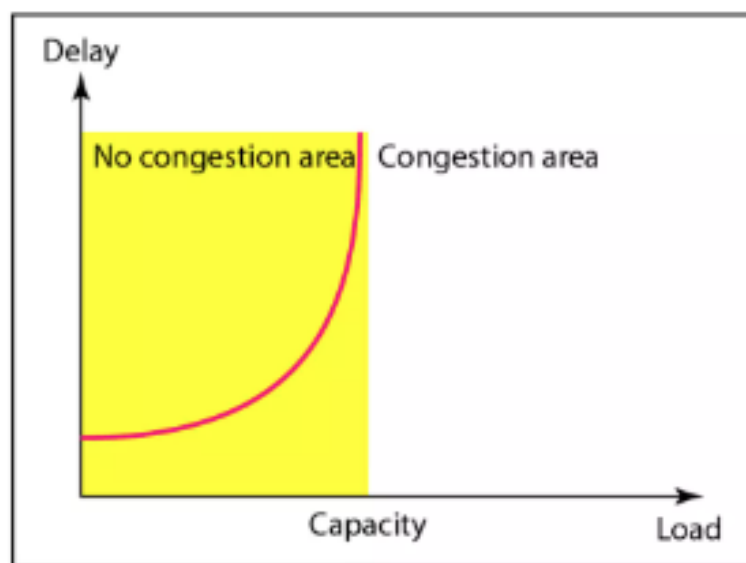
- When the **load is much less than the capacity of the network, the delay is at a minimum.** This minimum delay is composed of propagation delay and processing delay, both of which are negligible.
- When the **load reaches the network capacity, the delay increases sharply** because now need to add the waiting time in the queues to the total delay.

Throughput Versus Load

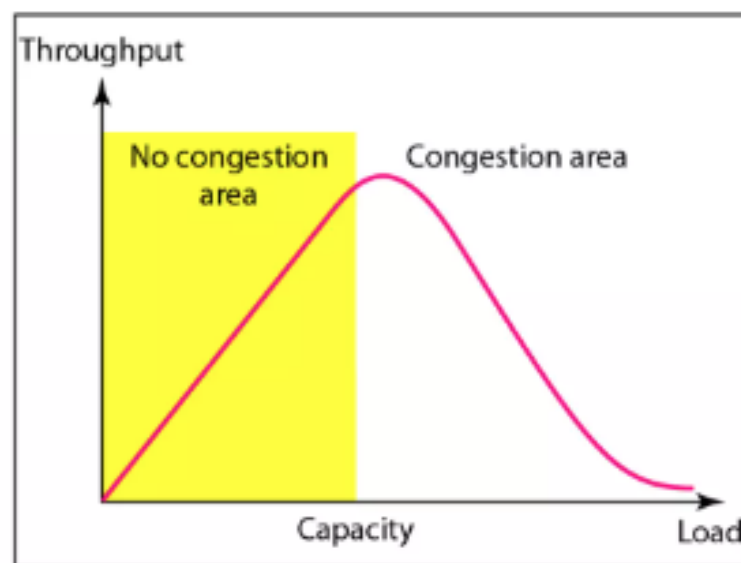
- **Throughput in a network is the number of packets passing through the network in a unit of time.** When the load is below the capacity of the network, the throughput increases proportionally with the load. After the load reaches the capacity, but instead the throughput declines sharply. **The reason is the discarding of packets by the routers.** When the load exceeds the capacity, the queues become full and the **routers have to discard some packets.** Discarding packet; does not reduce the number of packets in the network because the **sources retransmit the packets, using time-out mechanisms,** when the packets do not reach the destinations.

CONGESTION

Figure *Packet delay and throughput as functions of load*



a. Delay as a function of load

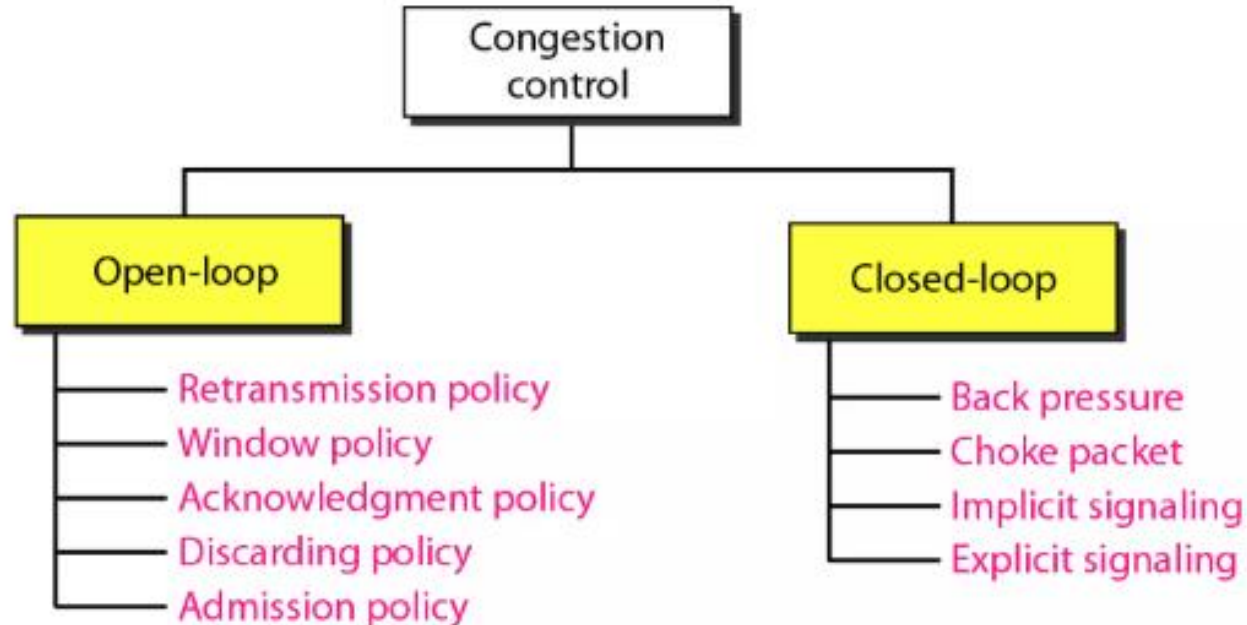


b. Throughput as a function of load

CONGESTION CONTROL

- Congestion control refers to techniques and mechanisms that can either prevent congestion, before it happens, or remove congestion, after it has happened.
- Congestion control mechanisms into two broad categories: open-loop congestion control (prevention) and closed-loop congestion control (removal)

Figure 24.5 *Congestion control categories*



CONGESTION CONTROL

Open-Loop Congestion Control

- In open-loop congestion control, policies are applied to **prevent** congestion before it happens. In these mechanisms, congestion control is handled by either the **source or the destination**.

Retransmission Policy

- Retransmission is sometimes unavoidable. If the sender feels that a sent packet is lost or corrupted, the packet needs to be retransmitted.
- Retransmission increases congestion in the network.
- A good retransmission policy can prevent congestion. The retransmission policy and the **retransmission timers must be designed to optimize efficiency and at the same time prevent congestion**.

Acknowledgment Policy

- If the receiver does not acknowledge every packet it receives, it may slow down the sender and help prevent congestion. Several approaches are used in this case.
- A receiver may send an acknowledgment only if it has a **packet to be sent or a special timer expires**.
- A receiver may decide to **acknowledge only N packets at a time**. Sending fewer acknowledgments means imposing less load on the network.

CONGESTION CONTROL

Open-Loop Congestion Control

Discarding Policy

- A **good discarding policy** by the routers may prevent congestion and at the same time may not harm the integrity of the transmission.
- For example, in audio transmission, if the policy is to **discard less sensitive packets when congestion is likely to happen, the quality of sound is still preserved and congestion is prevented.**

Admission Policy

- An admission policy, which is a quality-of-service mechanism, can also prevent congestion in virtual-circuit networks.
- **Switches in a flow first check the resource requirement of a flow before admitting it to the network.** A router can deny establishing a virtual circuit connection if there is congestion in the network or if there is a possibility of future congestion.

CONGESTION CONTROL

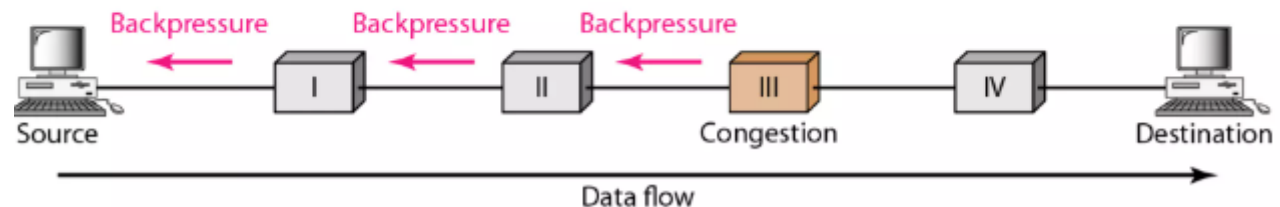
Closed-Loop Congestion Control

Closed-loop congestion control mechanisms try to alleviate congestion after it happens.

Backpressure

The technique of backpressure refers to a congestion control mechanism in which a congested node stops receiving data from the immediate upstream node or nodes. This may cause the upstream node or nodes to become congested, and they, in turn, reject data from their upstream nodes or nodes. And so on. Backpressure is a node-to-node congestion control that starts with a node and propagates, in the opposite direction of data flow, to the source.

Figure 24.6 *Backpressure method for alleviating congestion*



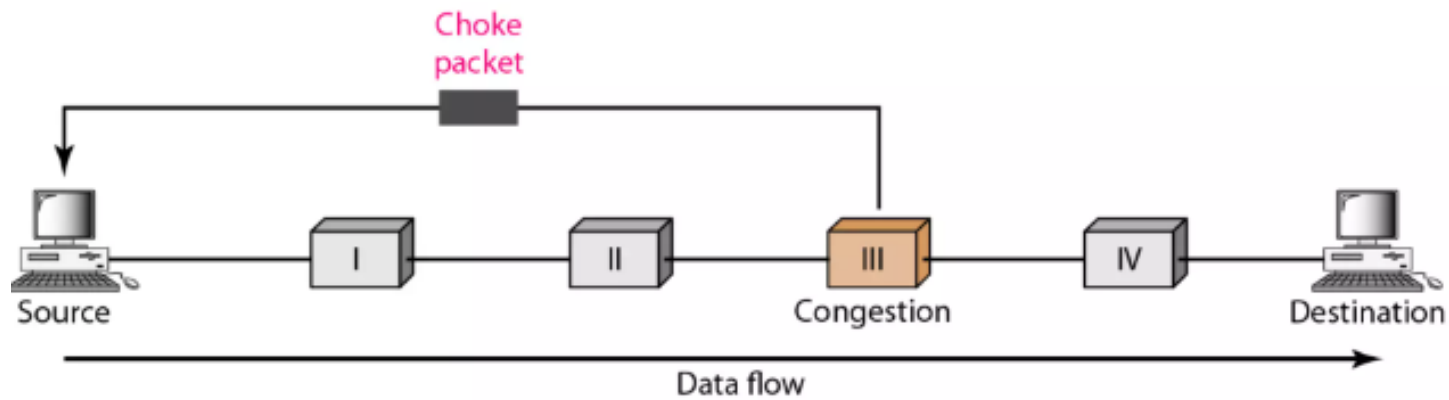
CONGESTION CONTROL

Closed-Loop Congestion Control

Choke Packet

A choke packet is a packet sent by a node to the source to inform it of congestion. In the choke packet method, the **warning is from the router, which has encountered congestion, to the source station directly**. The intermediate nodes through which the packet has traveled are not warned.

Figure 24.7 *Choke packet*



CONGESTION CONTROL

Closed-Loop Congestion Control

Implicit Signaling

In implicit signaling, there is **no communication between the congested node or nodes and the source**. The source guesses that there is a congestion somewhere in the network from other **symptoms**.

For example,

- **No acknowledgment** received for a while, one assumption is that the network is congested.
- The **delay in receiving an acknowledgment** is interpreted as congestion in the network; the source should slow down.

CONGESTION CONTROL

Closed-Loop Congestion Control

Explicit Signaling

- The node that experiences congestion can explicitly send a signal to the source or destination.
- In the explicit signaling method, the **signal is included in the packets that carry data.**
- Explicit signaling, can occur in either the forward or the backward direction.
- **Backward Signaling**
 - A **bit can be set in a packet moving in the direction opposite to the congestion.** This bit can **warn the source** that there is congestion and that it needs to slow down to avoid the discarding of packets.
- **Forward Signaling**
 - A **bit can be set in a packet moving in the direction of the congestion.** This bit can **warn the destination** that there is congestion. The receiver in this case can use policies, such as slowing down the acknowledgments, to alleviate the congestion.

Congestion Control in TCP

Congestion Window

- The sender window size is determined by the available buffer space in the receiver (**rwnd**).
- In addition to the receiver, the network is a second entity that determines the size of the sender's window.
- The sender has two pieces of information: the receiver-advertised window size and the congestion window size.
- The actual size of the window is the minimum of these two.
Actual window size = minimum (rwnd, cwnd)

Congestion Control in TCP

Congestion Policy

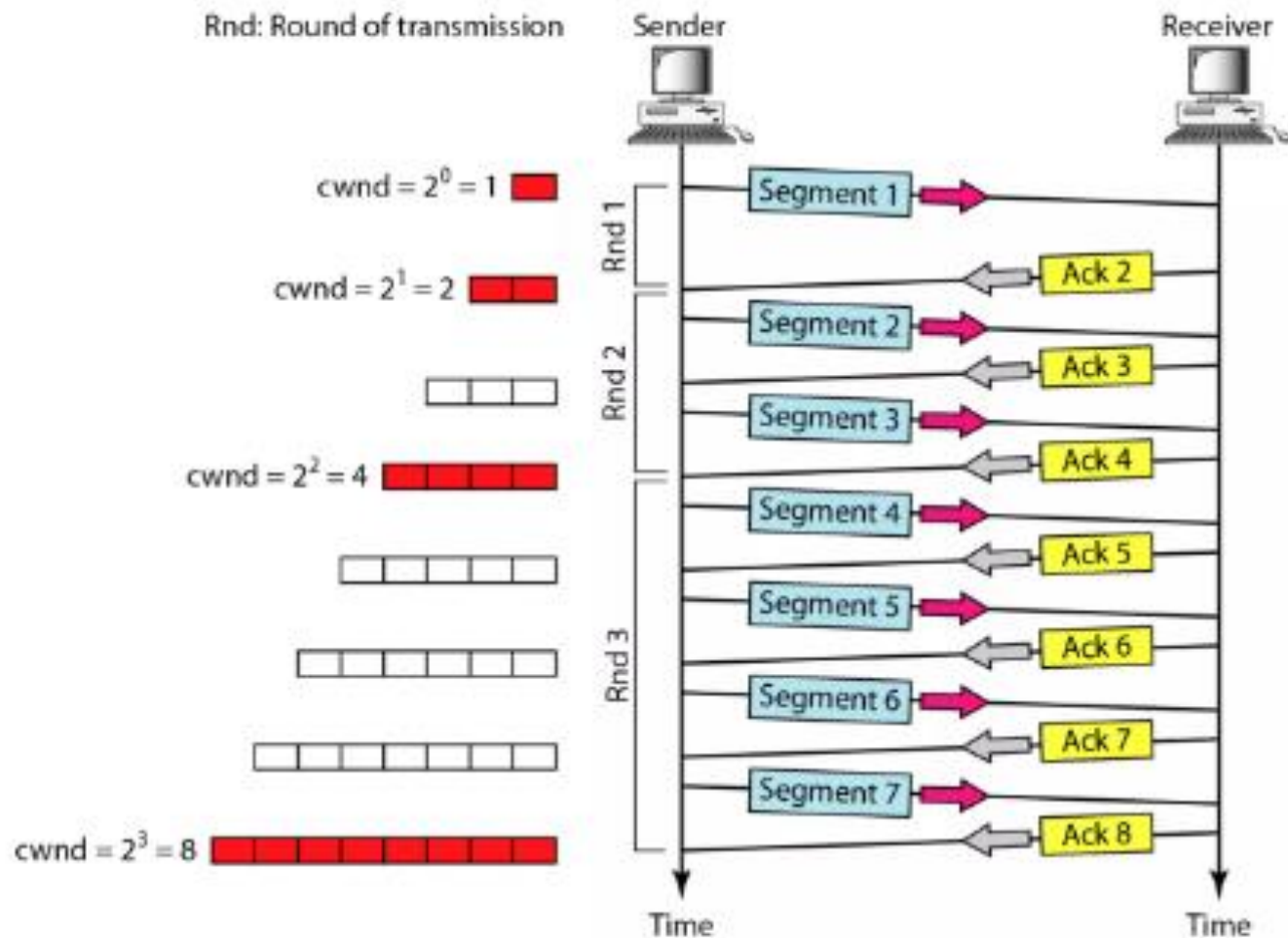
TCP's general policy for handling congestion is based on three phases: slow start, congestion avoidance, and congestion detection.

1. Slow-start phase

- The sender starts with a very slow rate of transmission, but increases the rate rapidly to reach a threshold.
- When the threshold is reached, the data rate is reduced to avoid congestion.
- Finally if congestion is detected, the sender goes back to the slow-start or congestion avoidance phase based on how the congestion is detected.

Congestion Control in TCP

Figure 24.8 *Slow start, exponential increase*



Congestion Control in TCP

- **Slow Start: Exponential Increase**
- This algorithm is based on the idea that the size of the **congestion window (cwnd)** starts with one maximum segment size (MSS). The MSS is determined during connection establishment.
- The size of the window increases one MSS each time an acknowledgment is received.
- **Three simplifications are made**
 - Segment numbers instead of byte numbers.
 - Assumed that rwnd is much higher than cwnd, so that the sender window size always equals cwnd.
 - Each segment is acknowledged individually.
- The sender starts with $cwnd = 1$ MSS. After receipt of the acknowledgment for segment 1, the size of the congestion window is increased by 1, which means that cwnd is now 2. Now two more segments can be sent. When each acknowledgment is received, the size of the window is increased by 1 MSS. When all seven segments are acknowledged, $cwnd = 8$.

Congestion Control in TCP

Slow Start: Exponential Increase

- Slow start cannot continue indefinitely. There must be a threshold to stop this phase.
- The **sender keeps track of a variable named ssthresh (slow-start threshold)**.
- When the size of window in bytes reaches this threshold, slow start stops and the next phase starts. In most implementations the value of ssthresh is 65,535 bytes.

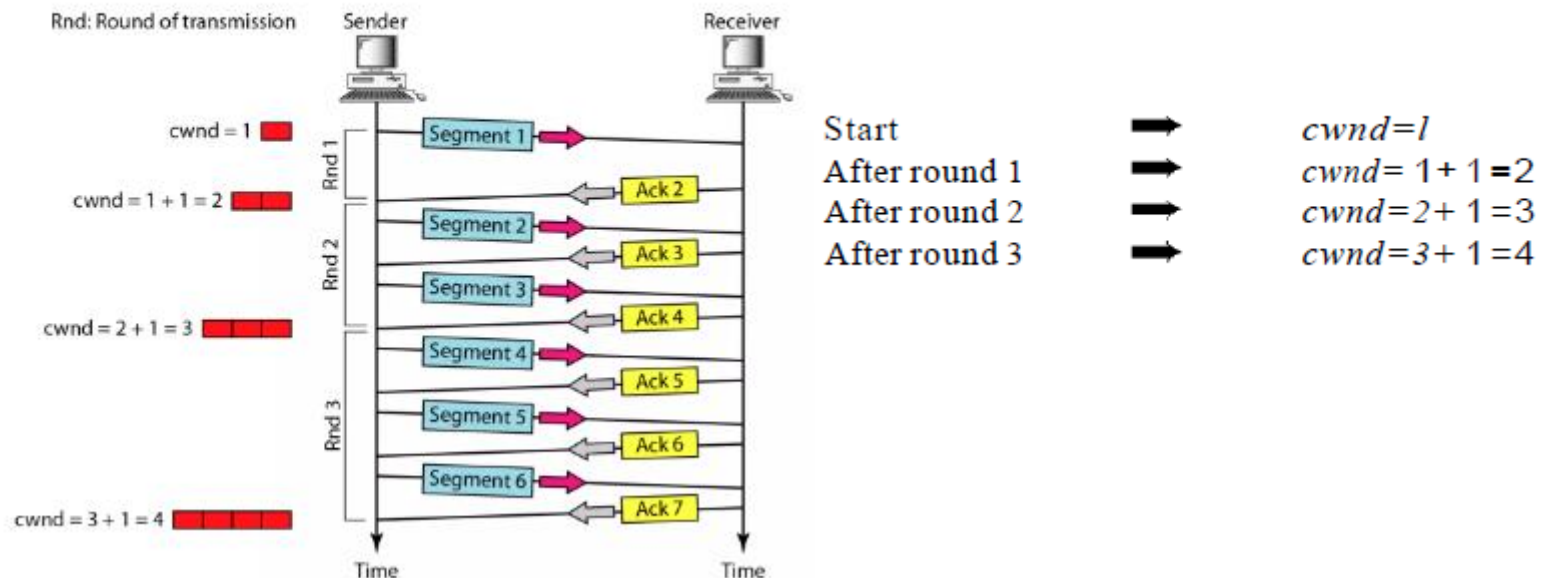
Start	➡	$cwnd=1$
After round 1	➡	$cwnd=2^1=2$
After round 2	➡	$cwnd=2^2=4$
After round 3	➡	$cwnd=2^3=8$

Congestion Control in TCP

Congestion Avoidance: Additive Increase

- If we start with the slow-start algorithm, the size of the congestion window increases exponentially. So congestion exponential grows.
- In congestion avoidance, When the size of the congestion window reaches the slow-start threshold, the slow-start phase stops and the additive phase begins.
- **Each time the whole window of segments is acknowledged (one round), the size of the congestion window is increased by 1.**

Figure 24.9 Congestion avoidance, additive increase



Congestion Control in TCP

Congestion Detection: Multiplicative Decrease:

- If congestion occurs, the congestion **window size must be decreased**. The only way the sender can guess that congestion has occurred is by the need to retransmit a segment.
- **Retransmission can occur in one of two cases:**
 1. When a timer times out
 2. When three ACKs are received.
- In both cases, the size of the threshold is dropped to one-half, a multiplicative decrease.
- Most TCP implementations have two reactions

Congestion Control in TCP

Time Out

- If a time-out occurs, there is a stronger possibility of congestion; a segment has probably been dropped in the network, and there is no news about the sent segments.

In this case TCP reacts strongly:

- a. It sets the value of the threshold to one-half of the current window size.
- b. It sets cwnd to the size of one segment.
- c. It starts the slow-start phase again.

Three ACKs are received

- This is a weaker possibility of congestion; a segment may have been dropped, but some segments after that may have arrived safely since three ACKs are received.

In this case, TCP has a weaker reaction:

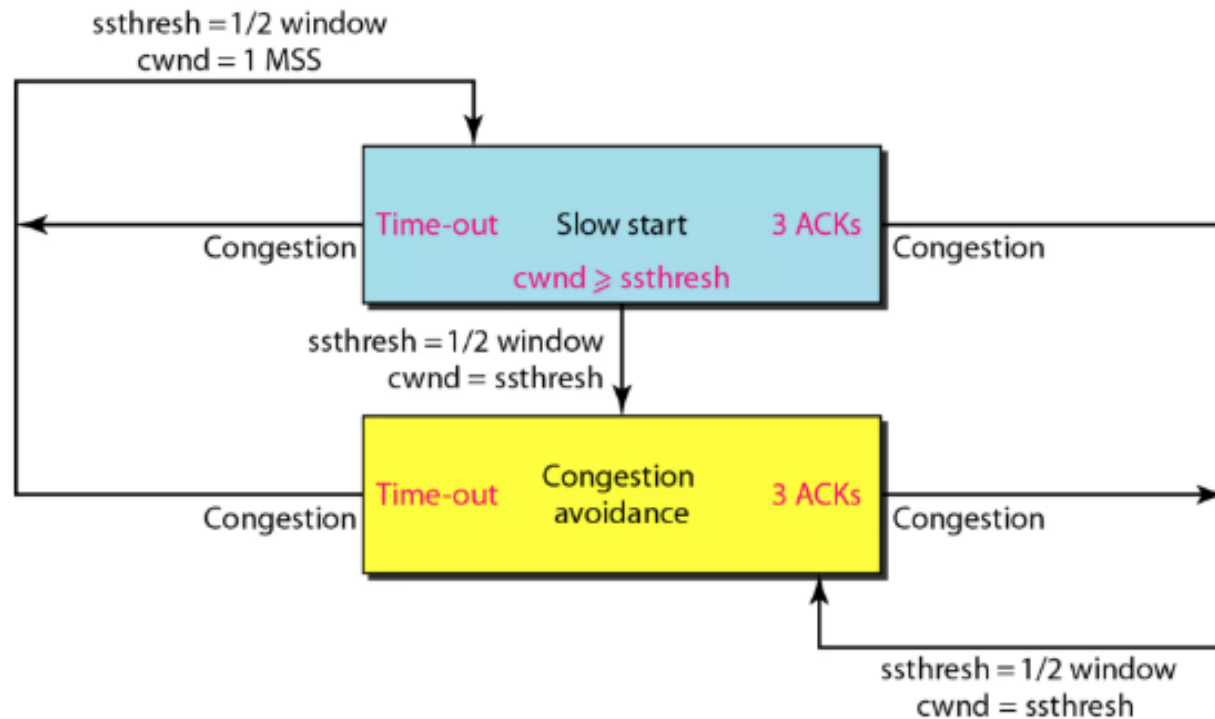
- a. It sets the value of the threshold to one-half of the current window size.
- b. It sets cwnd to the value of the threshold.
- c. It starts the congestion avoidance phase.

Congestion Control in TCP

An implementations reacts to congestion detection in one of the following ways:

- If detection is by time-out, a new slow-start phase starts.
- If detection is by three ACKs, a new congestion avoidance phase starts.

Figure 24.10 *TCP congestion policy summary*



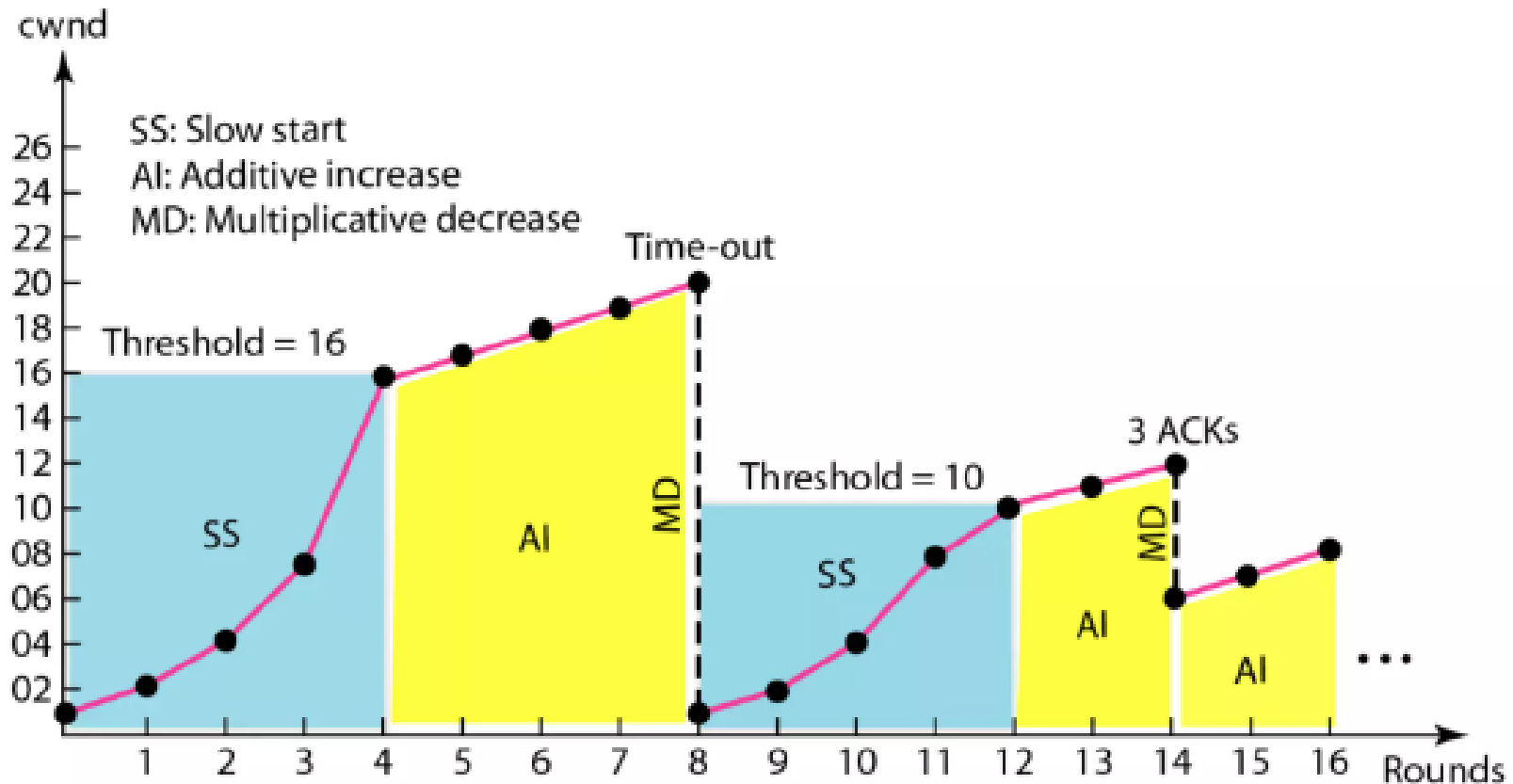
Congestion Control in TCP

Congestion Example:

- Assume that the maximum window size is 32 segments. The threshold is set to 16 segments (one-half of the maximum window size).
- In the slow-start phase the window size starts from 1 and grows exponentially until it reaches the threshold.
- After it reaches the threshold, the congestion avoidance (additive increase) procedure allows the window size to increase linearly until a timeout occurs or the maximum window size is reached.
- In Figure 24.11, the time-out occurs when the window size is 20. At this moment, the multiplicative decrease procedure takes over and reduces the threshold to one-half of the previous window size.
- The previous window size was 20 when the time-out happened so the new threshold is now 10.
- TCP moves to slow start again and starts with a window size of 1, and TCP moves to additive increase when the new threshold is reached.
- When the window size is 12, a three-ACKs event happens. The multiplicative decrease procedure takes over again.
- The threshold is set to 6 and TCP goes to the additive increase phase this time. It remains in this phase until another time-out or another three ACKs happen.

Congestion Control in TCP

Figure 24.11 *Congestion example*

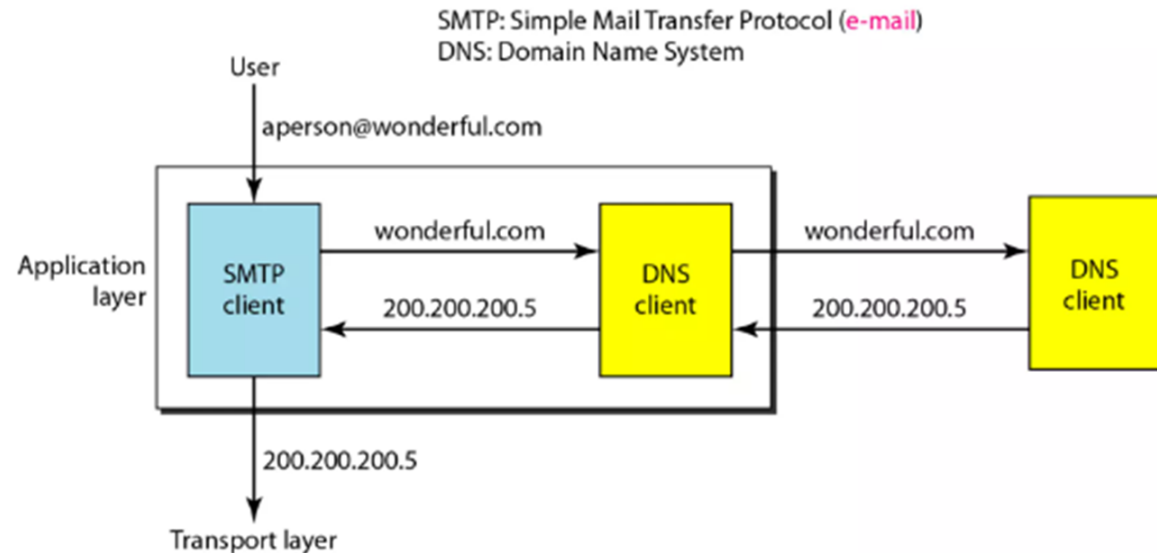


Application Layer

Domain Name System

- There are several applications in the application layer of the Internet model that follow the client/server paradigm.
- The client/server programs can be divided into two categories:
- Directly used by the user, such as e-mail, and those that support other application programs.
- The Domain Name System (DNS) is a supporting program that is used by other programs such as e-mail.

Figure 25.1 *Example of using the DNS service*



Domain Name System

- When the Internet was small, mapping was done by using a host file. The **host file** had only two columns: name and address.
- Store the entire host file in a single computer and allow access to this **centralized information** to every computer that needs mapping.
- **Divide this huge amount of information into smaller parts and store each part on a different computer.** In this method, the host that needs mapping can contact the closest computer holding the needed information. This method is used by the **Domain Name System (DNS)**

Domain Name System

Name Space

- To be unambiguous, the names assigned to machines must be carefully selected from a name space with complete control over the **binding between the names and IP addresses**.

Flat Name Space

- In a flat name space, a name is assigned to an address. A name in this space is a **sequence of characters** without structure.

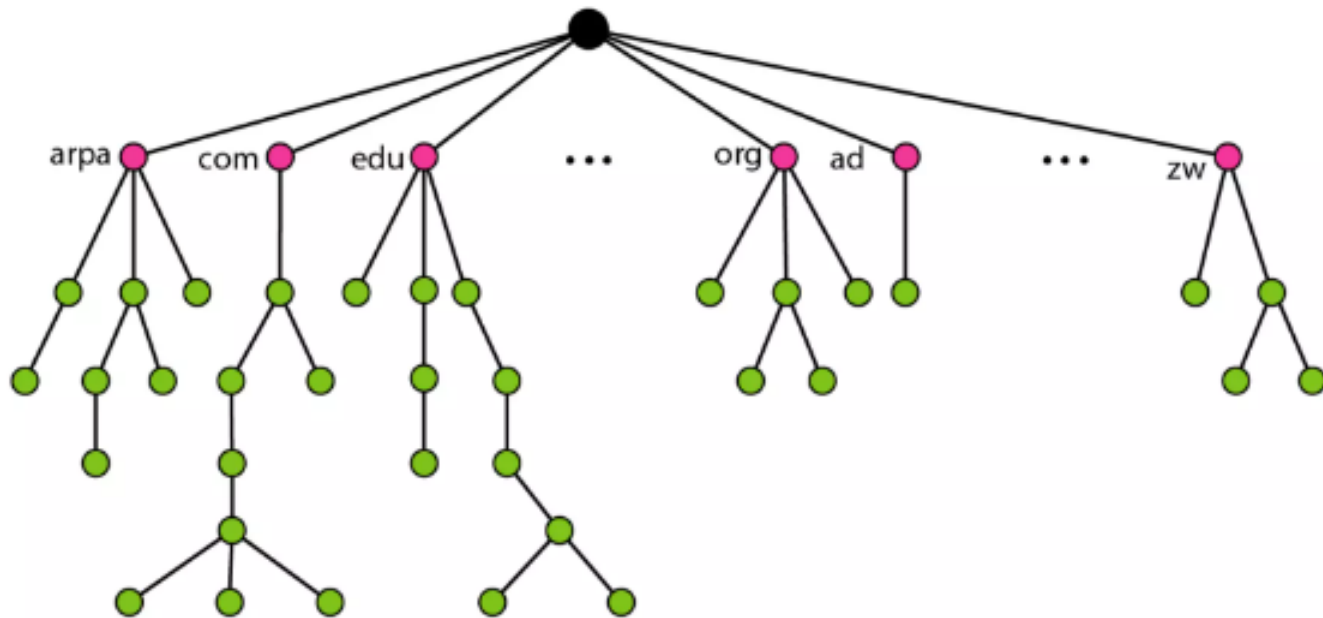
Hierarchical Name Space

- In a hierarchical name space, each name is made of several **parts**.
 1. Define the **nature** of the organization,
 2. Define the **name** of an organization,
 3. Define **departments** in the organization, and so on.

Domain Name Space

Names are defined in an inverted-tree structure with the root at the top. The tree can have only **128 levels: level 0 (root) to level 127**

Figure 25.2 *Domain name space*



Domain Name System

Label

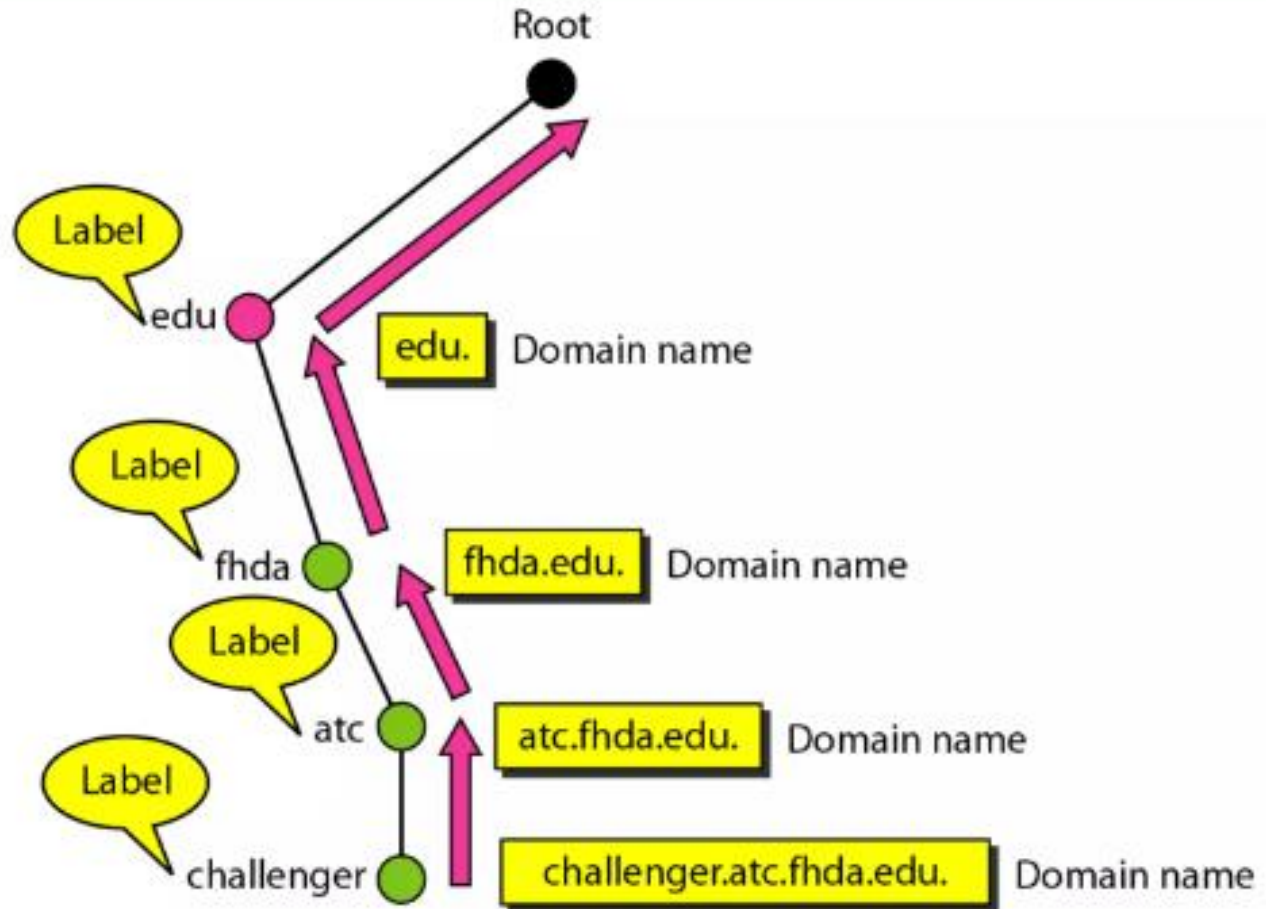
- Each node in the tree has a label, with a **maximum of 63 characters**.
- The root label is a null string (empty string).
- DNS requires that children of a node (nodes that branch from the same node) have different labels, which guarantees the uniqueness of the domain names.

Domain Name

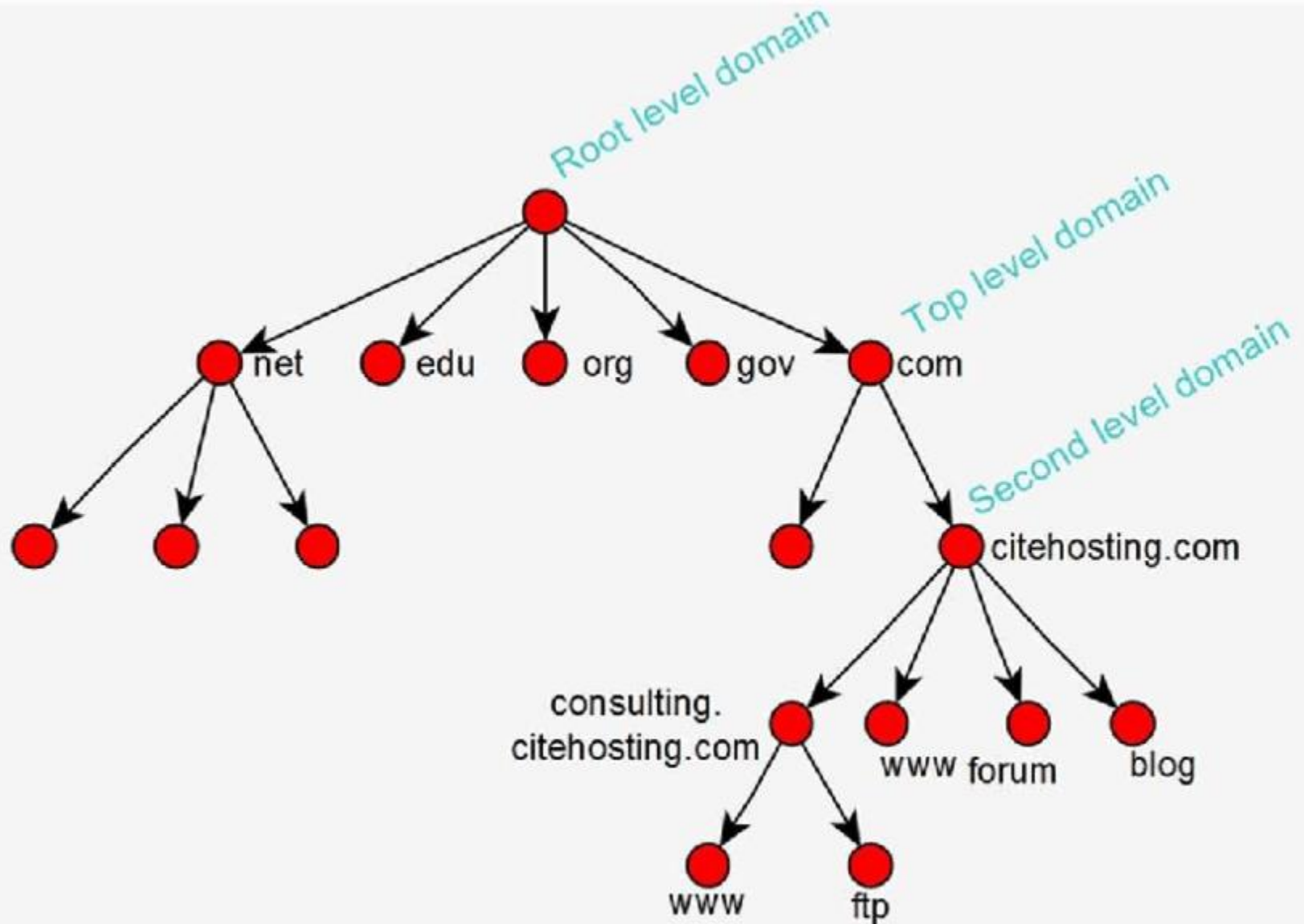
- Each node in the tree has a domain name. **A full domain name is a sequence of labels separated by dots (.).**
- The domain names are always **read from the node up to the root**.
- The last label is the label of the root (null). This means that a full domain name always ends in a null label, which means the **last character is a dot because the null string is nothing**

Domain Name System

Figure 25.3 *Domain names and labels*



Domain Name System



Domain Name System

Fully Qualified Domain Name

If a label is terminated by a null string, it is called a fully qualified domain name (FQDN). An FQDN is a domain name that contains the full name of a host. It contains all labels, from the most specific to the most general, that uniquely define the name of the host. For example, the domain name

challenger.atc.tbda.edu.

is the FQDN of a computer named challenger installed at the Advanced Technology Center (ATC) at De Anza College. A DNS server can only match an FQDN to an address. Note that the name must end with a null label, but because null means nothing, the label ends with a dot (.).

Domain Name System

Partially Qualified Domain Name

If a label is not terminated by a null string, it is called a partially qualified domain name (PQDN). A PQDN starts from a node, but it does not reach the root.

It is used when the name to be resolved belongs to the same site as the client. Here the resolver can supply the missing part, called the suffix, to create an FQDN.

For example, if a user at the jhda.edu. site wants to get the IP address of the challenger computer, he or she can define the partial name

challenger

The DNS client **adds the suffix atc.jhda.edu. before passing the address to the DNS server.**

The DNS client normally holds a list of suffixes. The following can be the list of suffixes at De Anza College. The null suffix defines nothing. This suffix is added when the user defines an FQDN.

Domain Name System

Figure 25.4 *FQDN and PQDN*

FQDN

challenger.atc.fhda.edu.
cs.hmme.com.
www.funny.int.

PQDN

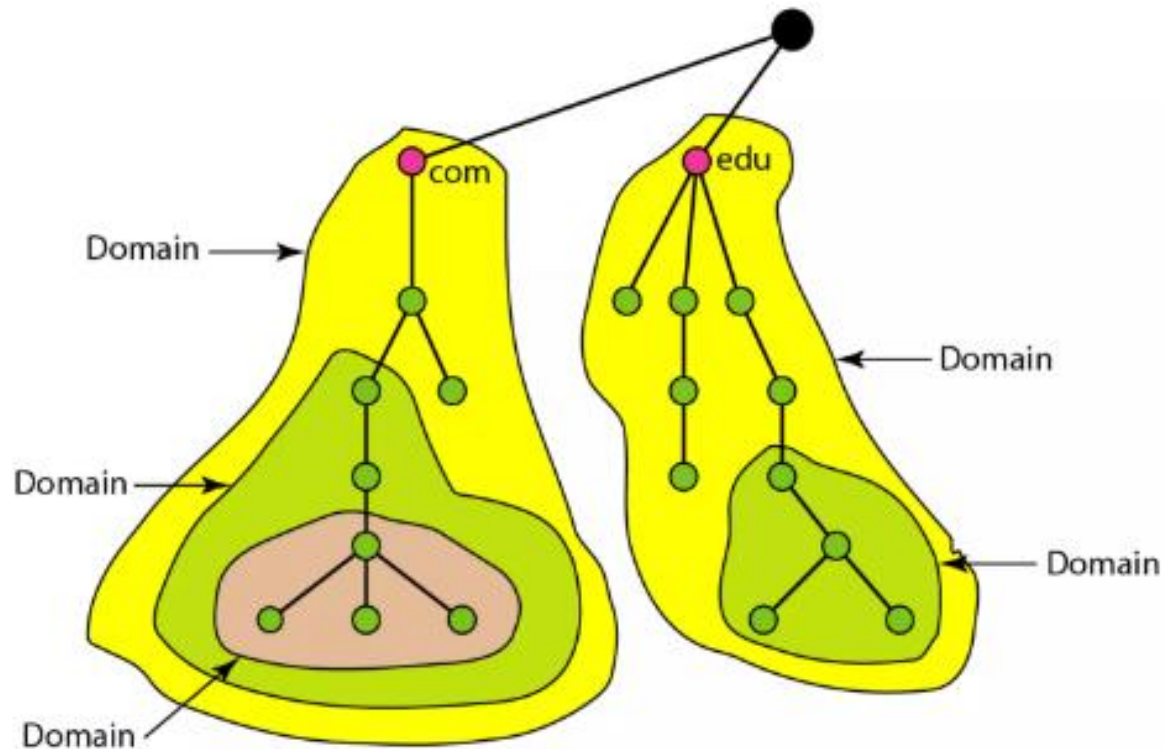
challenger.atc.fhda.edu
cs.hmme
www

Domain Name System

Domain

A domain is a subtree of the domain name space. The name of the domain is the domain name of the node at the top of the subtree.

Figure 25.5 *Domains*



DISTRIBUTION OF NAME SPACE

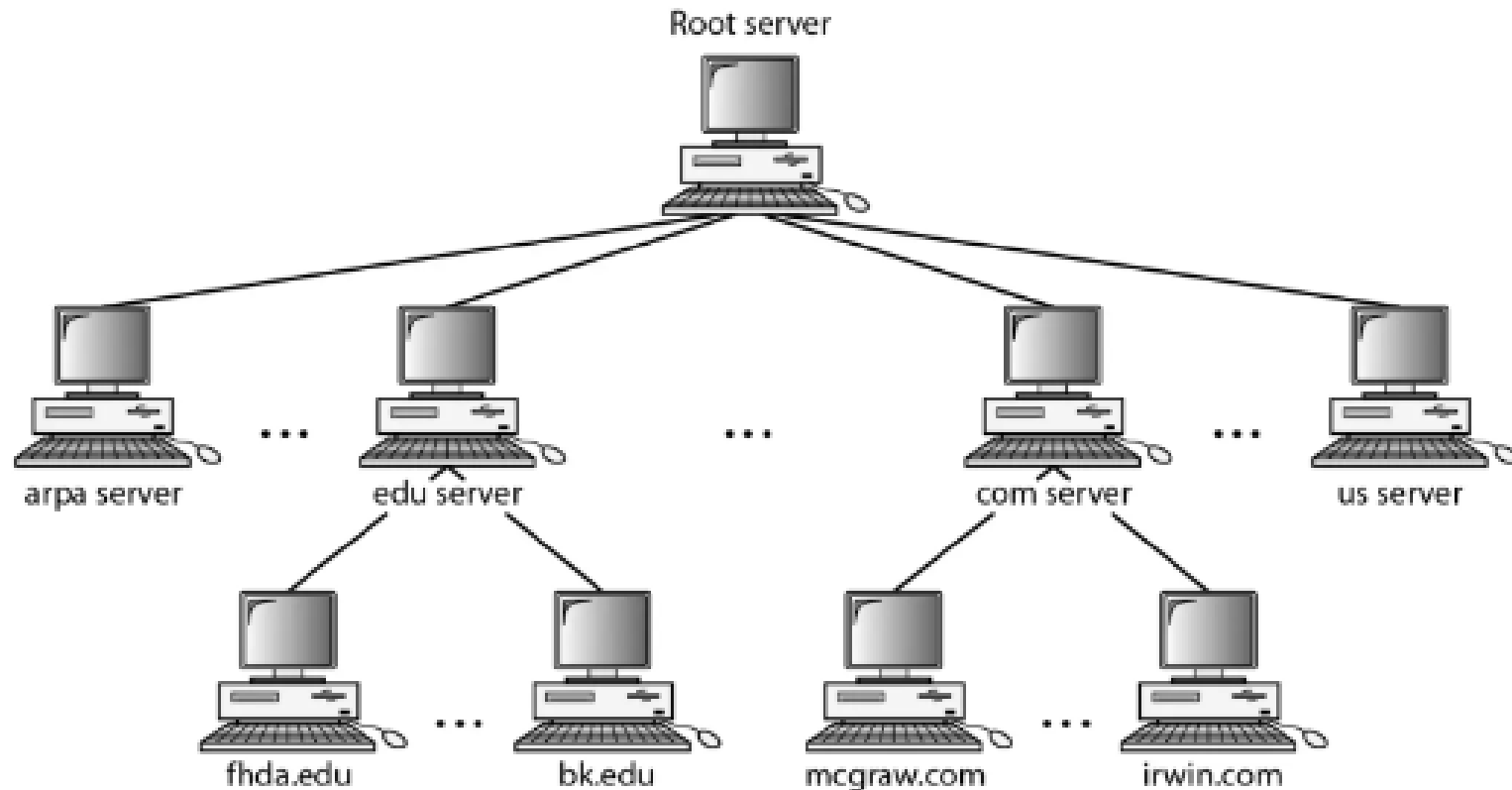
- It is **very inefficient** and also unreliable to have just one computer store such a huge amount of information.
- It is inefficient because responding to requests from all over the world places a **heavy load** on the system.
- It is **not unreliable** because any failure makes the data inaccessible.

Hierarchy of Name Servers

- The solution to these problems is to **distribute the information** among many computers called **DNS servers**.
- Divide the whole space into many domains based on the first level. In other words, we let the root stand alone and create as many domains (subtrees) as there are first-level nodes

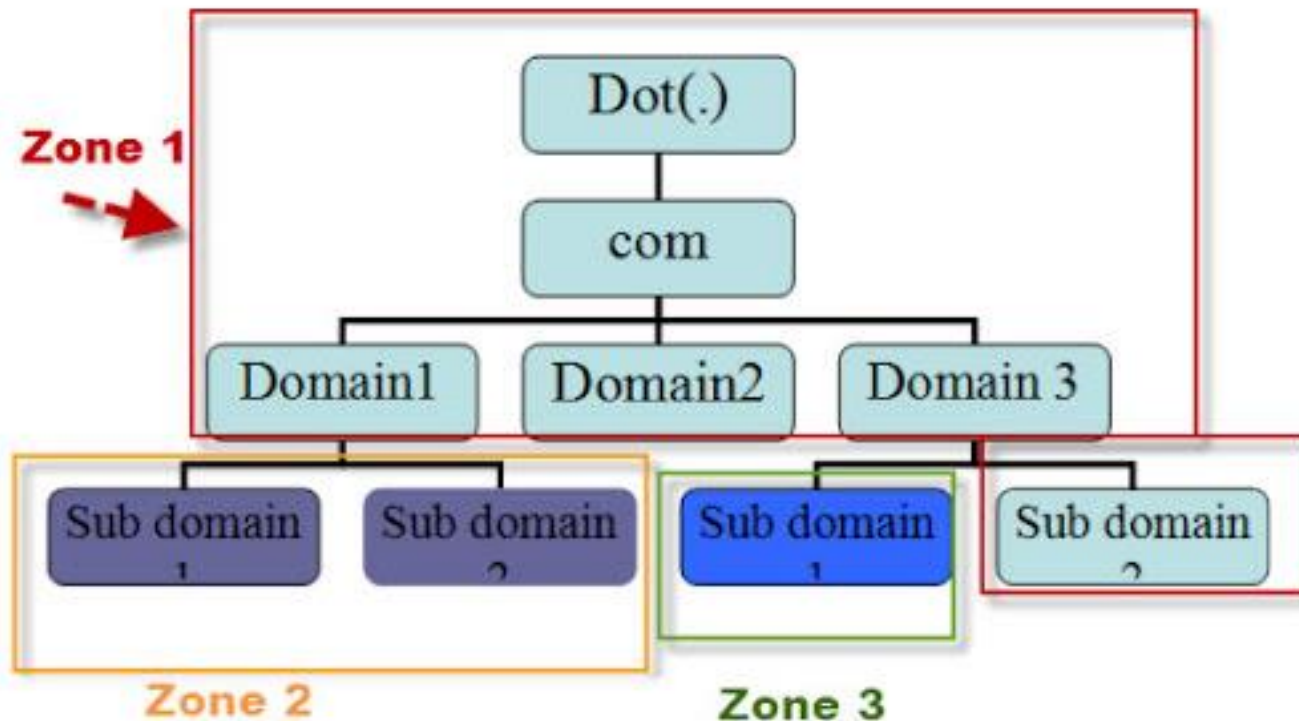
DISTRIBUTION OF NAME SPACE

Figure 25.6 *Hierarchy of name servers*



DISTRIBUTION OF NAME SPACE

DNS Zones Illustration



Notes:

Each zone has its own zone file

Zone 2 includes Domain 1 sub domains 1 and 2

Zone 3 only includes Domain 3 sub domain 1

DISTRIBUTION OF NAME SPACE

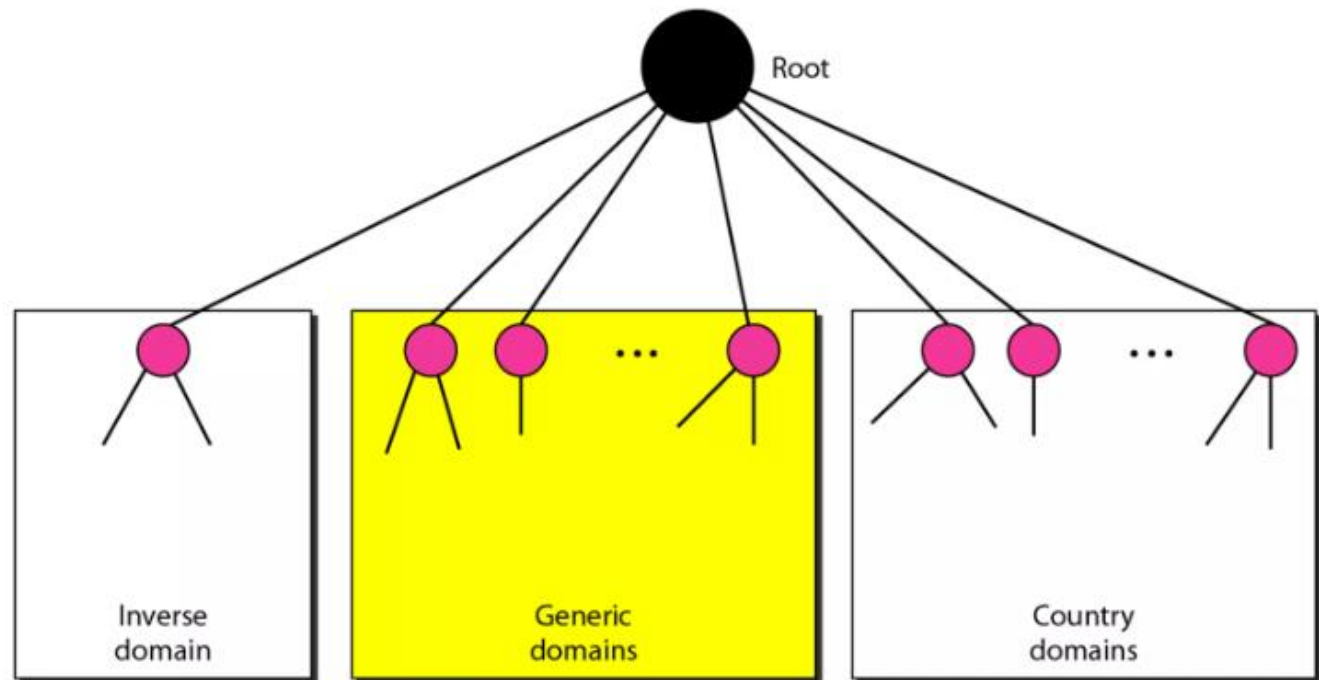
Zone

- Since the complete domain name hierarchy cannot be stored on a single server, it is divided among many servers.
- **What a server is responsible for or has authority over is called a zone.**
- **A zone as a contiguous part of the entire tree.** If a server accepts responsibility for a domain and does not divide the domain into smaller domains, the domain and the zone refer to the same thing. The server makes a database called a zone file and keeps all the information for every node under that domain.
- **If a server divides its domain into subdomains and delegates part of its authority to other servers, domain and zone refer to different things.** The information about the nodes in the subdomains is stored in the servers at the lower levels, with the original server keeping some sort of reference to these lower-level servers.

DNS IN THE INTERNET

- **DNS is a protocol** that can be used in different platforms.
- In the Internet, the domain name space (tree) is divided into three different sections: **generic domains, country domains, and the inverse domain**

Figure 25.8 *DNS IN THE INTERNET*

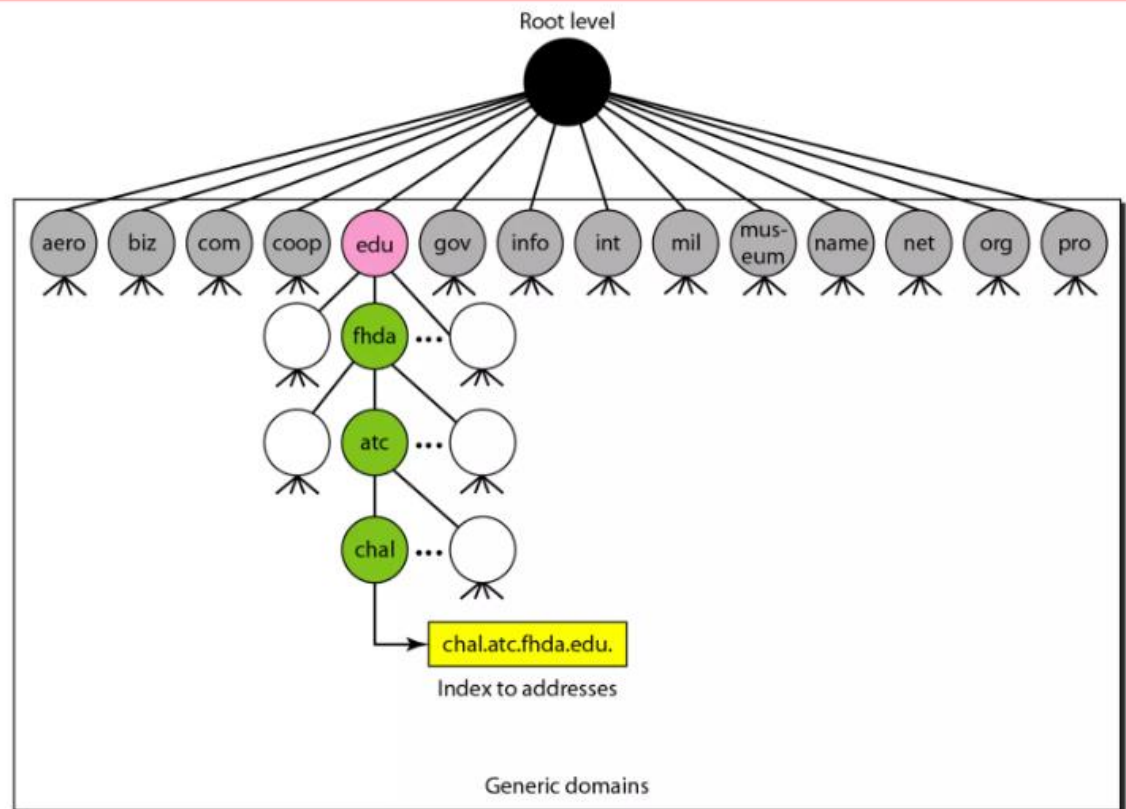


DNS IN THE INTERNET

Generic Domains

- The generic domains define **registered hosts according to their generic behavior**. Each node in the tree defines a domain, which is an index to the domain name space database

Figure 25.9 *Generic domains*



DNS IN THE INTERNET

Table 25.1 *Generic domain labels*

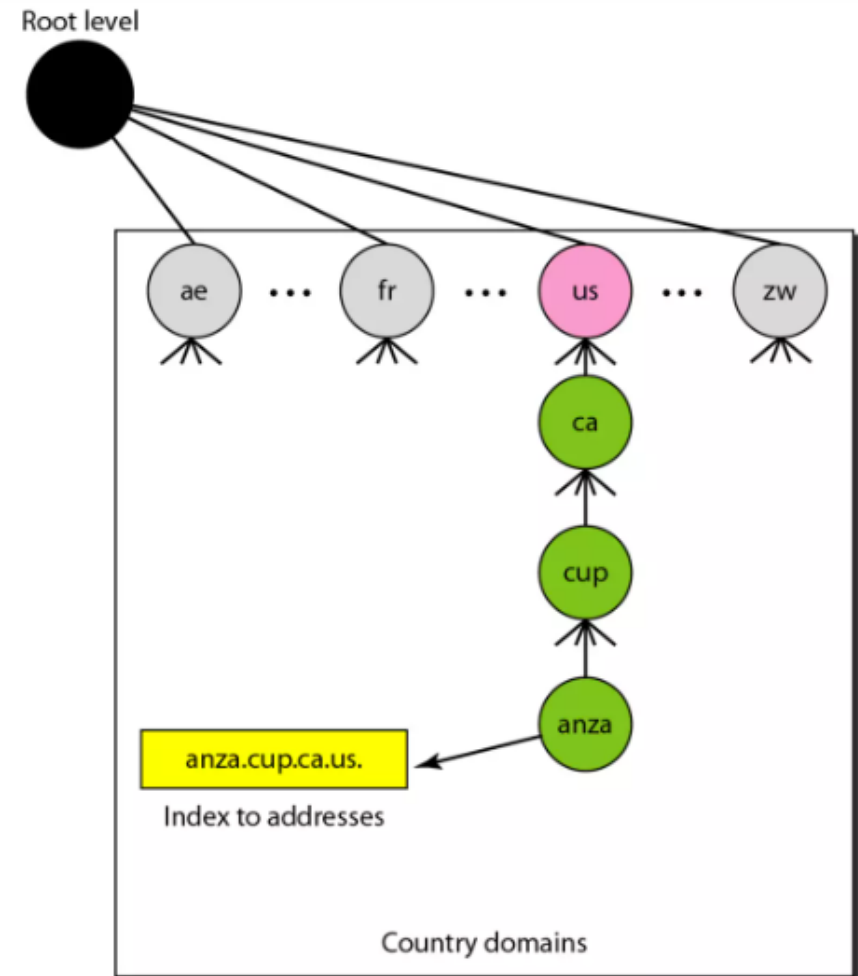
<i>Label</i>	<i>Description</i>
aero	Airlines and aerospace companies
biz	Businesses or firms (similar to “com”)
com	Commercial organizations
coop	Cooperative business organizations
edu	Educational institutions
gov	Government institutions
info	Information service providers
int	International organizations
mil	Military groups
museum	Museums and other nonprofit organizations
name	Personal names (individuals)
net	Network support centers
org	Nonprofit organizations
pro	Professional individual organizations

DNS IN THE INTERNET

Figure 25.10 *Country domains*

Country Domains

- The country domains section **uses two-character country abbreviations** (e.g., us for United States).
- **Second labels** can be organizational, or they can be more specific, national designations. The United States, for example, uses state abbreviations as a subdivision of us (e.g., ca.us.).



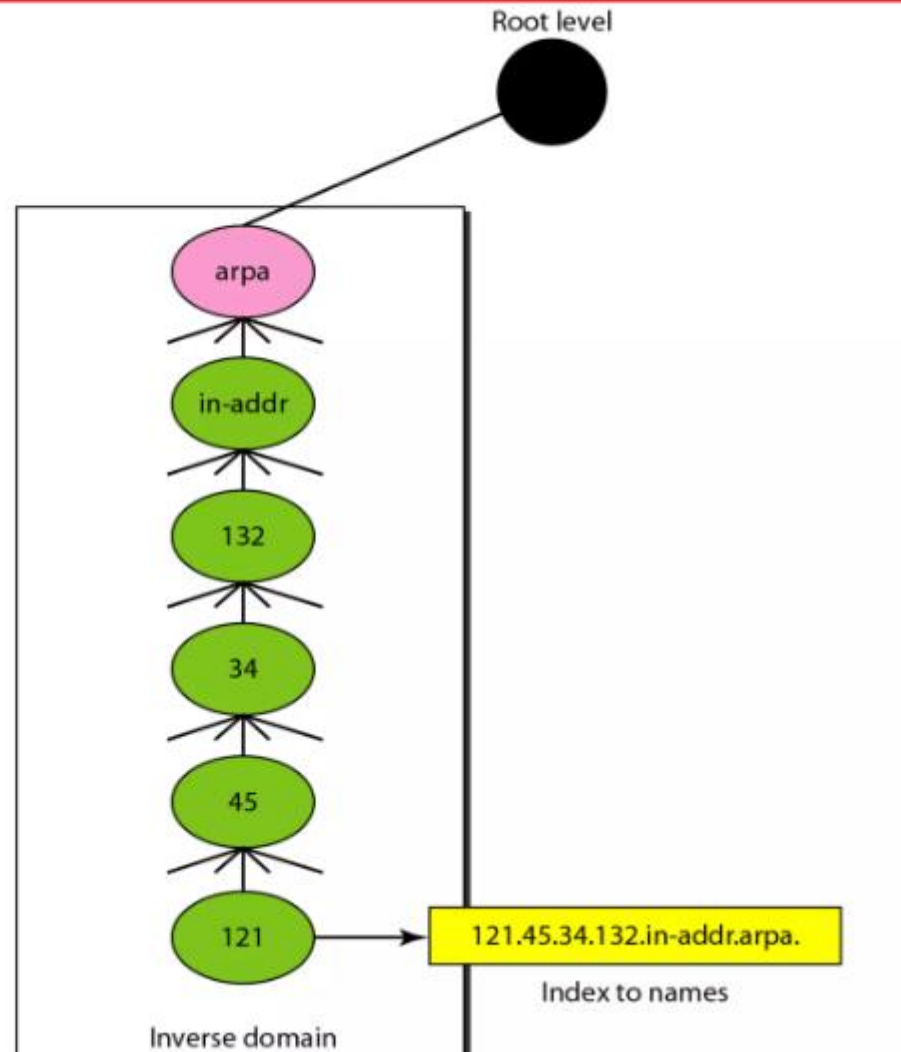
DNS IN THE INTERNET

Inverse Domain

- **The inverse domain is used to map an address to a name.**
- When a server has received a request from a client to do a task. Although the server has a file that contains a list of authorized clients, only the IP address of the client (extracted from the received IP packet) is listed. The server asks its resolver to send a query to the DNS server to map an address to a name to determine if the client is on the authorized list.
- This type of query is called an inverse or pointer (PTR) query. To handle a pointer query, the inverse domain is added to the domain name space with the first-level node called arpa (for historical reasons). The second level is also one single node named in-addr (for inverse address). The rest of the domain defines IP addresses

DNS IN THE INTERNET

Figure 25.11 *Inverse domain*



RESOLUTION

- Mapping a name to an address or an address to a name is called name-address resolution.

Resolver

- DNS is designed as a client/server application.
- A host that needs to map an address to a name or a name to an address calls a DNS client called a resolver.
- The resolver accesses the closest DNS server with a mapping request. If the server has the information, it satisfies the resolver; otherwise, it either refers the resolver to other servers or asks other servers to provide the information.
- After the resolver receives the mapping, it interprets the response to see if it is a real resolution or an error, and finally delivers the result to the process that requested it.

RESOLUTION

Mapping Names to Addresses

- The resolver gives a domain name to the server and asks for the corresponding address. In this case, the server checks the generic domains or the country domains to find the mapping.
- If the domain name is from the generic domains section, the resolver receives a domain name such as "chal.atc.jhda.edu.". The query is sent by the resolver to the local DNS server for resolution. If the local server cannot resolve the query, it either refers the resolver to other servers or asks other servers directly.
- If the domain name is from the country domains section, the resolver receives a domain name such as "ch.jhda.cu.ca.us.". The procedure is the same.

RESOLUTION

Mapping Addresses to Names

- A client can send an IP address to a server to be mapped to a domain name.
- DNS uses the inverse domain.
- In the request, the IP address is reversed and the two labels in-addr and arpa are appended to create a domain acceptable by the inverse domain section.
- For example, if the resolver receives the IP address 132.34.45.121, the resolver first inverts the address and then adds the two labels before sending. The domain name sent is "121.45.34.132.in-addr.arpa." which is received by the local DNS and resolved

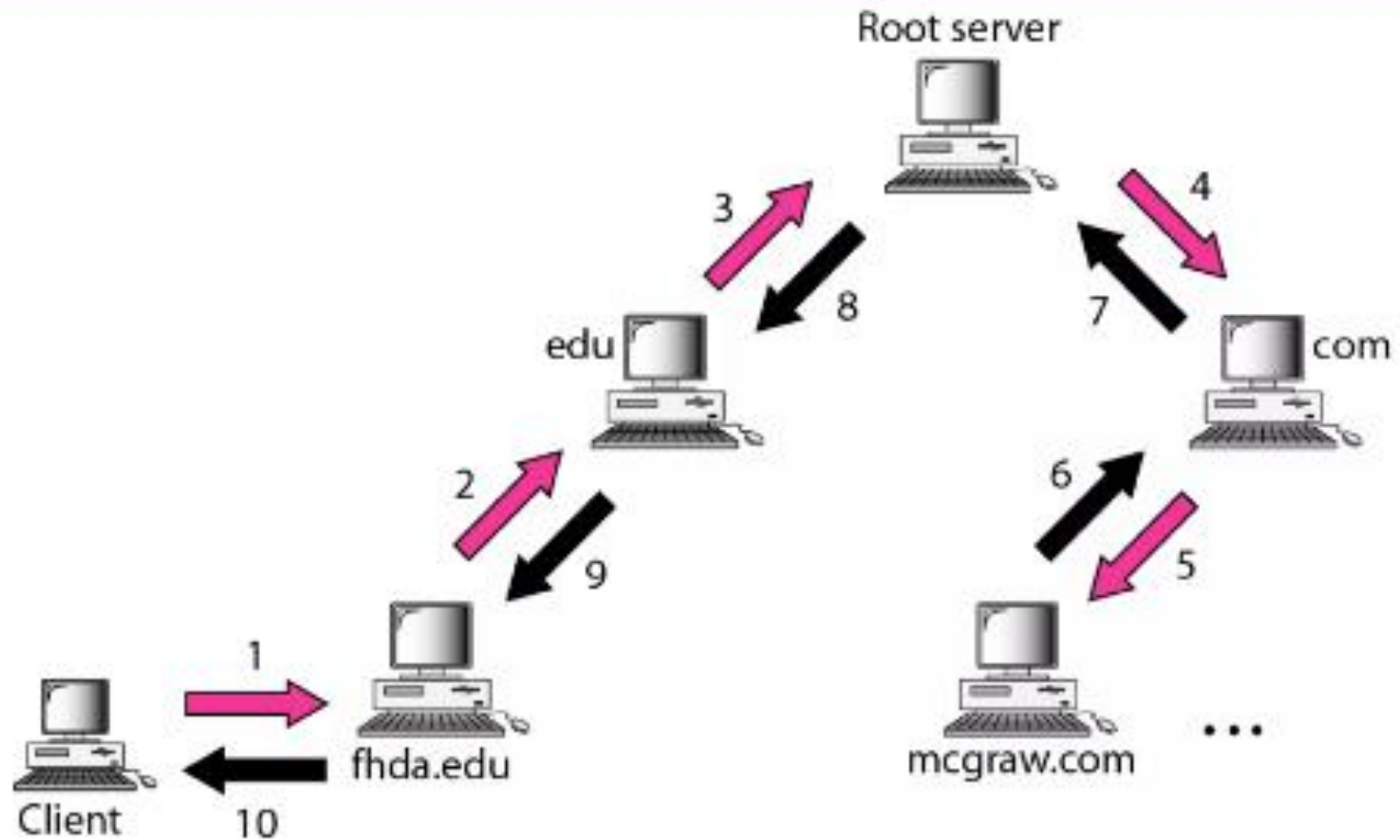
RESOLUTION

Recursive Resolution

- The resolver expects the server to supply the final answer. If the server is the authority for the domain name, it checks its database and responds.
- If the server is not the authority, it sends the request to another server (the parent usually) and waits for the response.
- If the parent is the authority, it responds; otherwise, it sends the query to yet another server.
- When the query is finally resolved, the response travels back until it finally reaches the requesting client. This is called recursive resolution

RESOLUTION

Figure 25.12 *Recursive resolution*



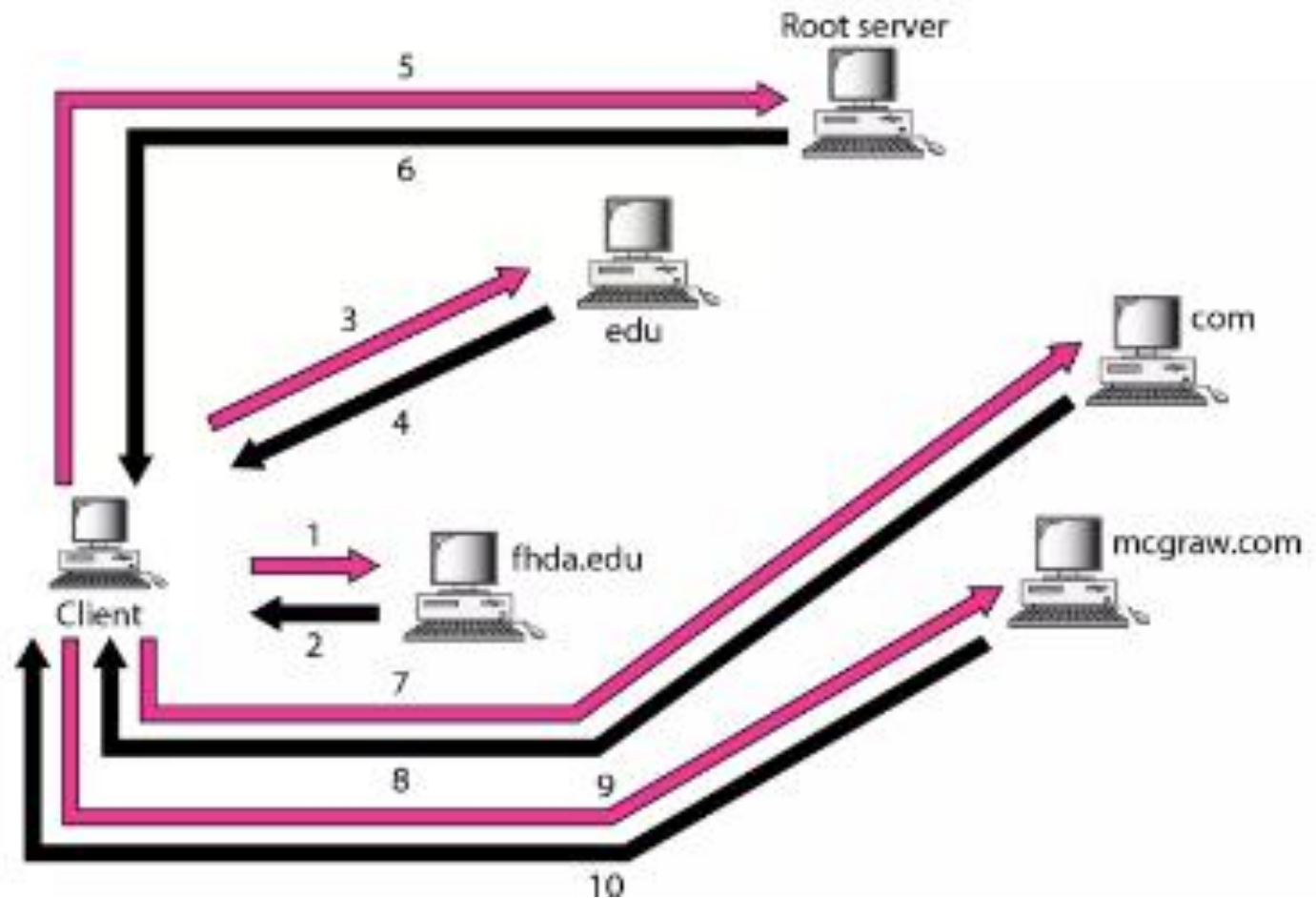
RESOLUTION

Iterative Resolution

- If the client does not ask for a recursive answer, the mapping can be done iteratively.
- If the server is an authority for the name, it sends the answer. If it is not, it returns (to the client) the IP address of the server that it thinks can resolve the query.
- The client is responsible for repeating the query to this second server. If the newly addressed server can resolve the problem, it answers the query with the IP address; otherwise, it returns the IP address of a new server to the client.
- Now the client must repeat the query to the third server. This process is called iterative resolution because the client repeats the same query to multiple servers. In Figure 25.13 the client queries four servers before it gets an answer from the mcgraw.com server

RESOLUTION

Figure 25.13 *Iterative resolution*



RESOLUTION

Caching

- Each time a server receives a query for a name that is not in its domain, it needs to search its database for a server IP address. Reduction of this search time would increase efficiency.
- DNS handles this with a mechanism called caching. When a server asks for a mapping from another server and receives the response, it stores this information in its cache memory before sending it to the client.
- If the same or another client asks for the same mapping, it can check its cache memory and solve the problem. However

REMOTE LOGGING

- In the Internet, users may want to run application programs at a remote site and create results that can be transferred to their local site.
- Remote Logging is client/server program that lets a user access any application program on a remote computer; in other words, allow the user to log on to a remote computer.
- After logging on, a user can use the services available on the remote computer and transfer the results back to the local computer.

TELNET

- TELNET is a **client/server** application program
- Its abbreviation is **TErminaL NETwork**.
- It is the standard TCP/IP protocol for virtual terminal service as proposed by the International Organization for Standards (**ISO**).
- TELNET enables the establishment of a connection to a remote system in such a way that the local terminal appears to be a terminal at the remote system.

TELNET

Timesharing Environment

- TELNET was designed at a time when most operating systems, such as **UNIX**, were operating in a timesharing environment.
- In such an environment, a large computer supports **multiple users**.
- The interaction between a user and the computer occurs through a **terminal, which is usually a combination of keyboard, monitor, and mouse**.

TELNET

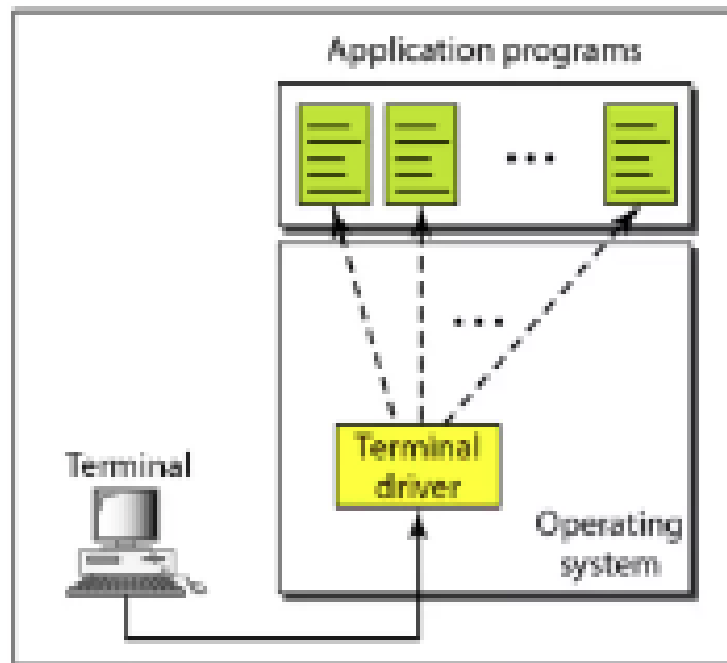
Logging

- In a timesharing environment, users are part of the system with some right to access resources.
- Each authorized user has an identification and a password.
- Two Types of Login
 - Local log-in
 - Remote log-in

TELNET

1. Local log-in

- When a user logs into a local timesharing system, it is called **local log-in**.
- As a user types at a terminal or at a workstation running a terminal emulator, the keystrokes are accepted by the **terminal driver**.
- The terminal driver passes the characters to the **operating system**.
- The operating system, in turn, interprets the combination of characters and **invokes the desired application program or utility**.

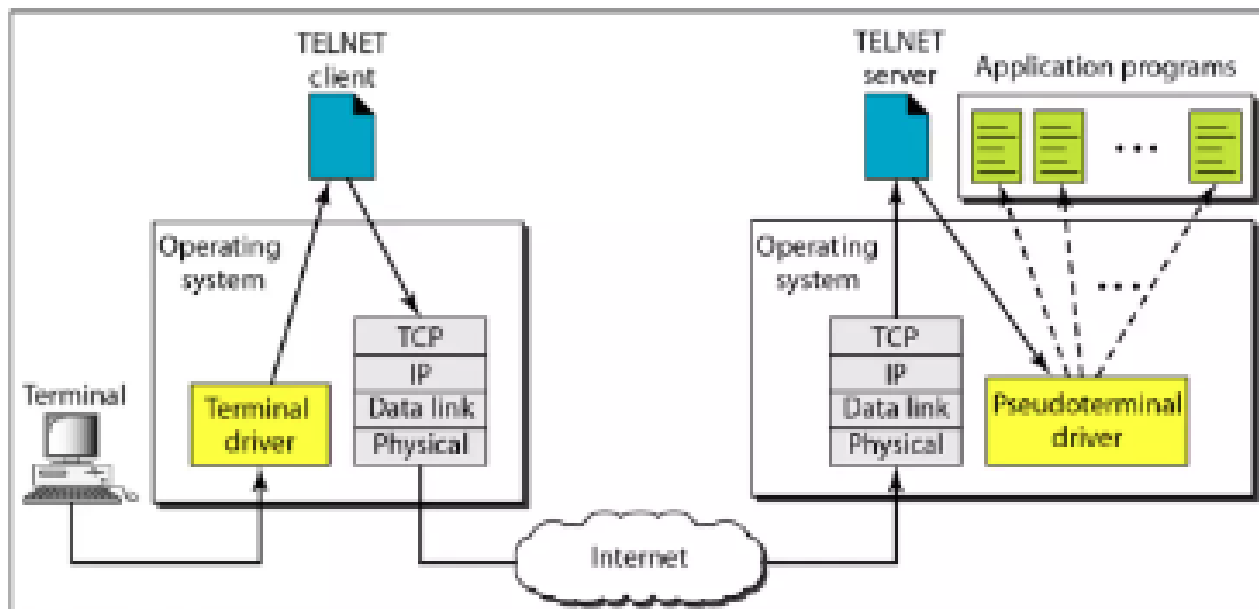


a. Local log-in

TELNET

2. Remote log-in

- When a user wants to access an application program or utility located on a remote machine, she performs **remote log-in**.
- Here the **TELNET client and server programs** come into use.
- The user sends the keystrokes to the terminal driver, where the **local operating system accepts the characters but does not interpret them**.
- The characters are sent to the TELNET client, which transforms the characters to a universal character set called **network virtual terminal (NVT) characters** and delivers them to the **local TCP/IP protocol stack**.



b. Remote log-in

TELNET

- The commands or text, in NVT form, travel through the Internet and arrive at the TCP/IP stack at the remote machine.
- Here the characters are delivered to the operating system and passed to the TELNET server, which changes the characters to the corresponding characters understandable by the remote computer.
- A piece of software called a pseudoterminal driver is added which pretends that the characters are coming from a terminal.
- The operating system then passes the characters to the appropriate application program.

TELNET

Network Virtual Terminal

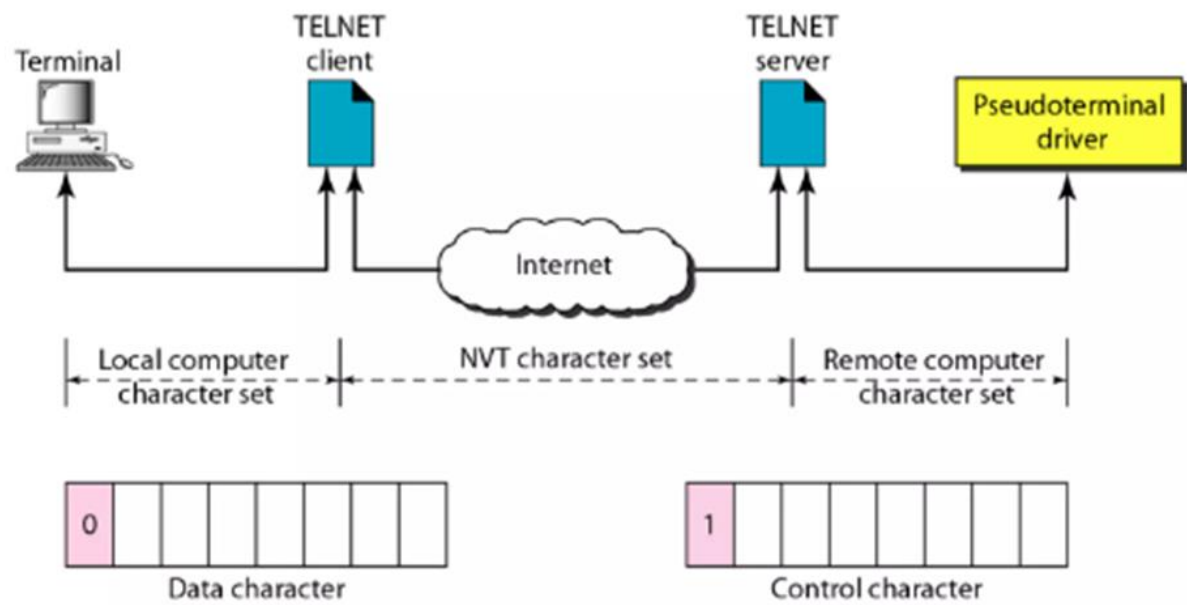
- Every computer and its operating system accept a special combination of characters as tokens.
- **For example**, the end-of-file token in a computer running the DOS operating system is Ctrl+z, while the UNIX operating system recognizes Ctrl+d.
- Internet is made of **heterogeneous systems**.
- TELNET solves this problem by defining a **universal interface called the network virtual terminal (NVT) character set**.
- Via this interface, the client **TELNET translates** characters (data or commands) that come from the local terminal into NVT form and delivers them to the network.
- The server TELNET, on the other hand, translates data and commands from NVT form into the form acceptable by the remote computer

TELNET

Network Virtual Terminal

- NVT Character Set NVT uses two sets of characters, one for data and the other for control.
- For data, NVT is an 8-bit character set in which the 7 lowest-order bits are the same as ASCII and the highest-order bit is 0.
- To send control characters between computers (from client to server or vice versa), NVT uses an 8-bit character set in which the highest-order bit is set to 1.

Figure 26.2 *Concept of NVT*



TELNET

Table 26.1 *Some NVT control characters*

<i>Character</i>	<i>Decimal</i>	<i>Binary</i>	<i>Meaning</i>
EOF	236	11101100	End of file
EOR	239	11101111	End of record
SE	240	11110000	Suboption end
NOP	241	11110001	No operation
DM	242	11110010	Data mark
BRK	243	11110011	Break
IP	244	11110100	Interrupt process
AO	245	11110101	Abort output
AYT	246	11110110	Are you there?
EC	247	11110111	Erase character
EL	248	11111000	Erase line
GA	249	11111001	Go ahead
SB	250	11111010	Suboption begin
WILL	251	11111011	Agreement to enable option
WONT	252	11111100	Refusal to enable option
DO	253	11111101	Approval to option request
DONT	254	11111110	Denial of option request
IAC	255	11111111	Interpret (the next character) as control

TELNET

Embedding

- TELNET uses only one TCP connection. The server uses the well-known port 23, and the client uses an ephemeral port.
- The same connection is used for sending both data and control characters.
- TELNET accomplishes this by embedding the control characters in the data stream.
- To distinguish data from control characters, each sequence of control characters is preceded by a special control character called Interpret As Control (IAC).
- For example, imagine a user wants a server to display a file (file 1) on a remote server.
- The command is

cat file1

- Suppose the name of the file has been mistyped (filea instead of file1).
- The user uses the backspace key to correct this situation.

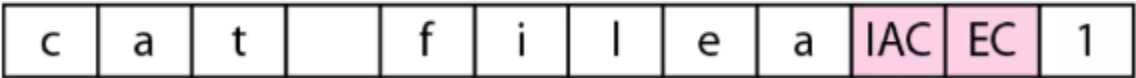
cat filea<backspace>1

TELNET

Embedding

- In the default implementation of TELNET, the user cannot edit locally; the editing is done at the remote server. The backspace character is translated into two remote characters (IAC EC), which are embedded in the data and sent to the remote server. What is sent to the server is shown in Figure 26.3.

Figure 26.3 *An example of embedding*



Typed at the remote terminal

ELECTRONIC MAIL

- One of the **most popular Internet services** is electronic mail (e-mail).
- At the beginning of the Internet era, the messages sent by electronic mail were **short and consisted of text only**; they let people exchange quick memos.
- Today, electronic mail is much more complex. It allows a message to include **text, audio, and video**. It **also allows one message to be sent to one or more recipients**.
- Study the **general architecture of an e-mail system** including the **three main components: user agent, message transfer agent, and message access agent**.

ELECTRONIC MAIL

- **Architecture**
- The architecture of e-mail is explained using **four scenarios**.
- Start with the simplest situation and add complexity as we proceed.
- The **fourth scenario is the most common in the exchange of email**.

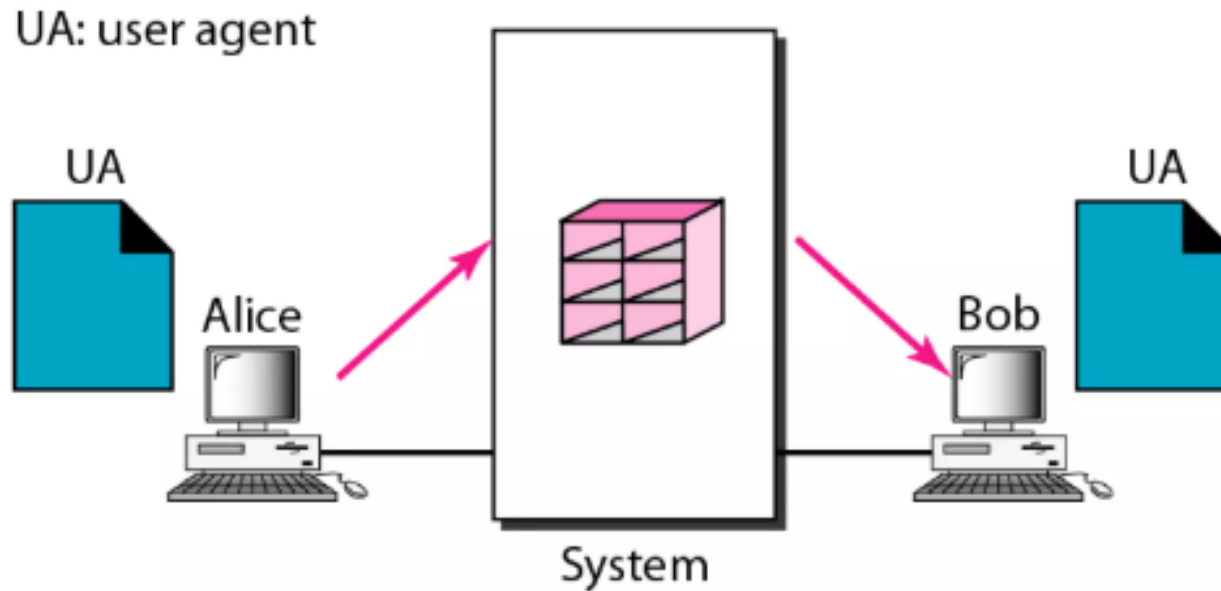
ELECTRONIC MAIL

First Scenario

- In the first scenario, **the sender and the receiver of the e-mail are users (or application programs) on the same system**; they are directly connected to a shared system.
- The administrator has created one mailbox for each user where the received messages are stored.
- A **mailbox is part of a local hard drive**, a special file with permission restrictions.
- Only the owner of the mailbox has access to it. When Alice, a user, needs to send a message to Bob, another user, Alice runs a user agent (UA) program to prepare the message and store it in Bob's mailbox.
- The message has the sender and recipient mailbox addresses (names of files). Bob can retrieve and read the contents of his mailbox at his convenience, using a user agent.
- Figure 26.6 shows the concept. This is similar to the traditional memo exchange between employees in an office.

ELECTRONIC MAIL

Figure 26.6 *First scenario in electronic mail*



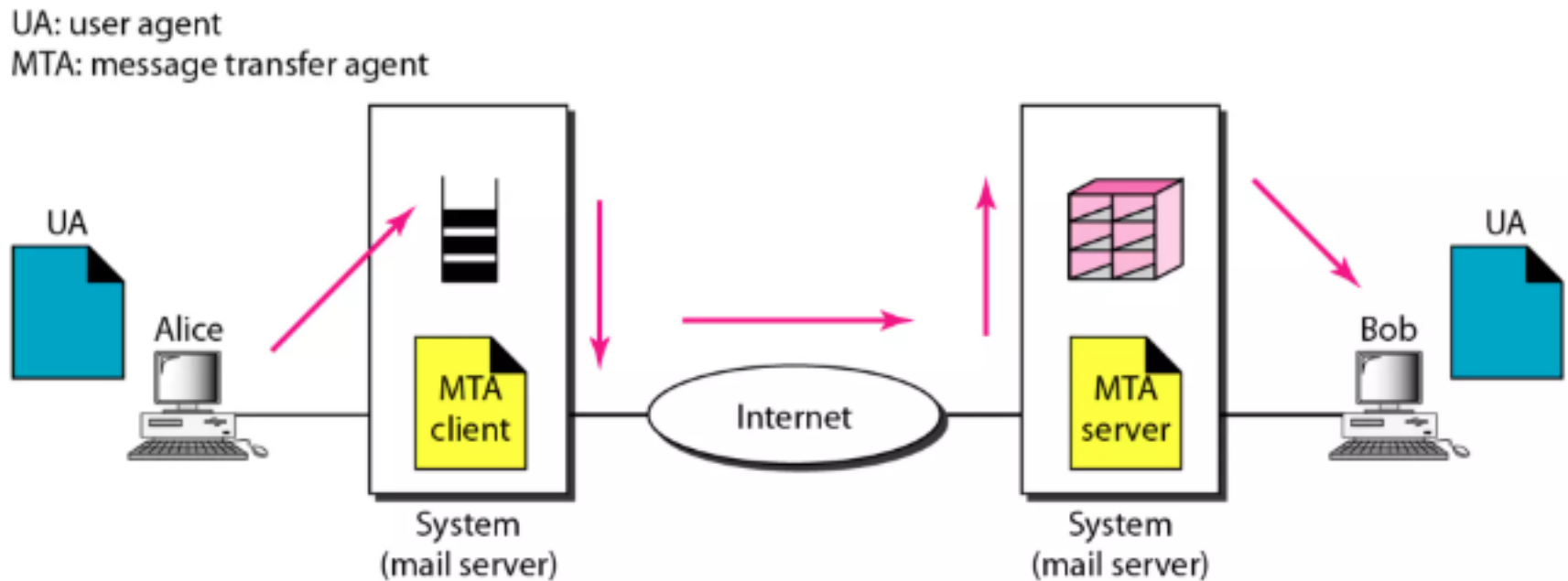
ELECTRONIC MAIL

Second Scenario

- In the second scenario, **the sender and the receiver of the e-mail are users (or application programs) on two different systems.**
- The message needs to be sent over the Internet.
- **Need user agents (UAs) and message transfer agents (MTAs).**
- Alice needs to use a user agent program to send her message to the system at her own site. The system (sometimes called the mail server) at her site uses a queue to store messages waiting to be sent.
- Bob also needs a user agent program to retrieve messages stored in the mailbox of the system at his site. The message, however, needs to be sent through the Internet from Alice's site to Bob's site.
- Like most client/server programs on the Internet, the server needs to run all the time because it does not know when a client will ask for a connection.
- The client, on the other hand, can be alerted by the system when there is a message in the queue to be sent.

ELECTRONIC MAIL

Figure 26.7 *Second scenario in electronic mail*



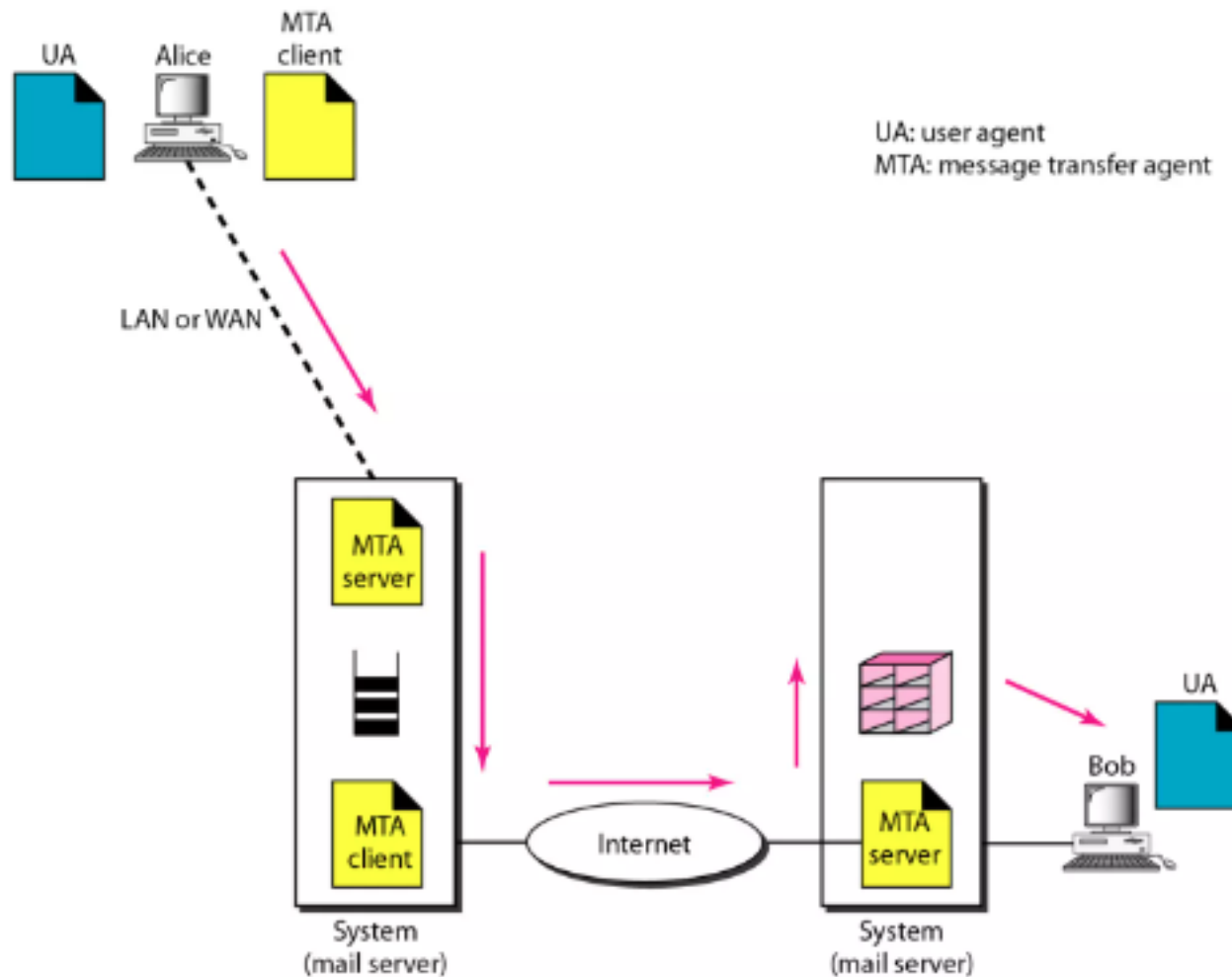
ELECTRONIC MAIL

Third Scenario

- In the third scenario, Bob, as in the second scenario, is directly connected to his system.
- Alice, however, is separated from her system. Either **Alice is connected to the system via a point-to-point WAN, such as a dial-up modem, a DSL, or a cable modem; or she is connected to a LAN in an organization that uses one mail server for handling e-mails-all** users need to send their messages to this mail server.
- Alice needs a user agent to prepare her message. She then needs to send the message through the LAN or WAN. This can be done through a pair of message transfer agents (client and server).
- Whenever Alice has a message to send, she calls the user agent which, in turn, calls the MTA client. **The MTA client establishes a connection with the MTA server on the system, which is running all the time.**
- The system at Alice's site queues all messages received. It then uses an MTA client to send the messages to the system at Bob's site; the system receives the message and stores it in Bob's mailbox.

ELECTRONIC MAIL

Figure 26.8 *Third scenario in electronic mail*



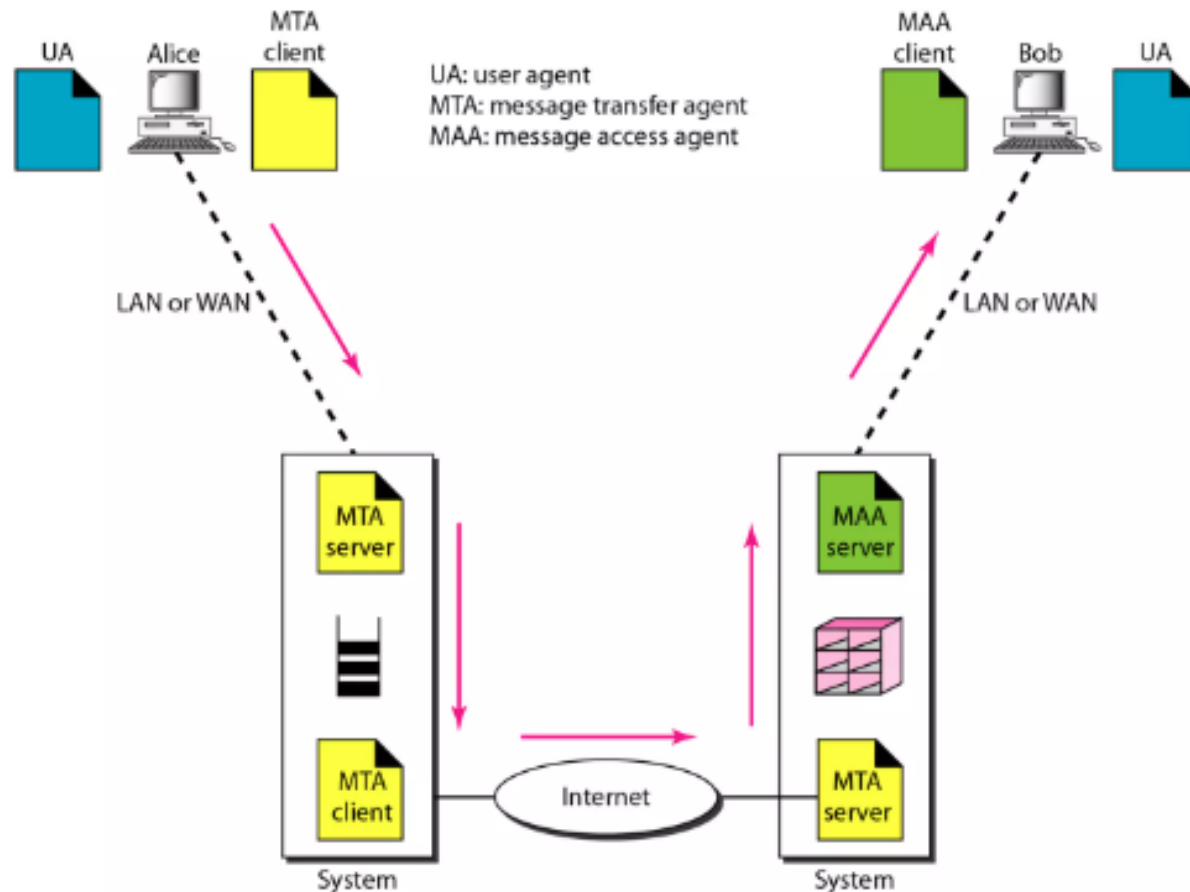
ELECTRONIC MAIL

Fourth Scenario

- In the fourth and most common scenario, **Bob is also connected to his mail server by a WAN or a LAN.** After the message has arrived at Bob's mail server, Bob needs to retrieve it.
- Here, we need another set of client/server agents, which we call message access agents (MAAs). Bob uses an MAA client to retrieve his messages.
- The client sends a request to the MAA server, which is running all the time, and requests the transfer of the messages.
- **There are two important points here.**
 1. Bob cannot bypass the mail server and use the MTA server directly. To use MTA server directly, Bob would need to run the MTA server all the time because he does not know when a message will arrive. This implies that Bob must keep his computer on all the time if he is connected to his system through a LAN.
 2. Bob needs another pair of client/server programs: message access programs. This is so because an **MTA client/server program is a push program:** the client pushes the message to the server. Bob **needs a pull program. The client needs to pull the message from the server.**

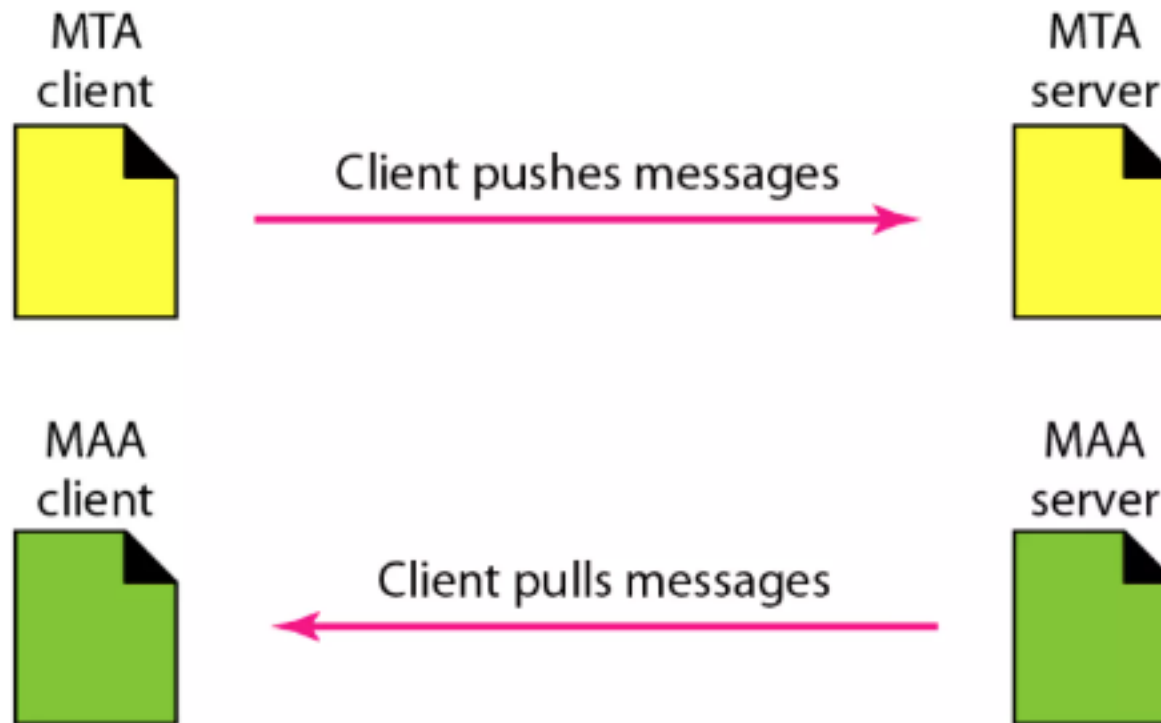
ELECTRONIC MAIL

Figure 26.9 *Fourth scenario in electronic mail*



ELECTRONIC MAIL

Figure 26.10 *Push versus pull in electronic email*

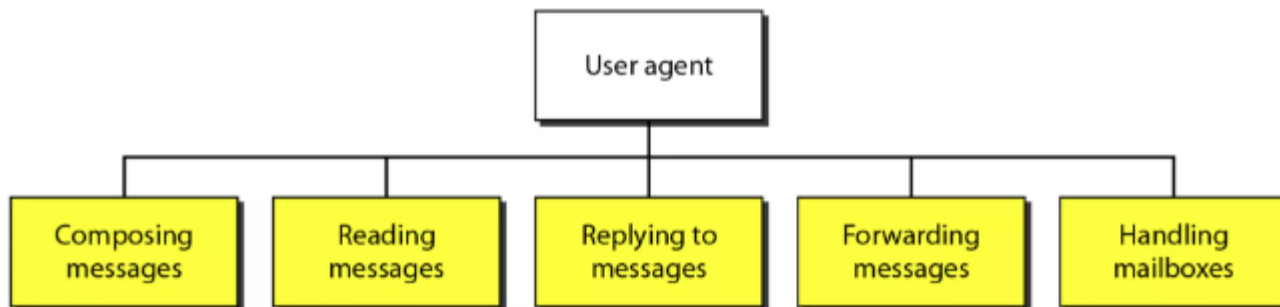


ELECTRONIC MAIL

User Agent

- The first component of an electronic mail system is the user agent (UA). It provides service to the user to make the process of sending and receiving a message easier.
- **Services Provided by a User Agent**
 - A user agent is a software package (program) that composes, reads, replies to, and forwards messages. It also handles mailboxes.

Figure 26.11 *Services of user agent*



ELECTRONIC MAIL

Composing Messages

- A user agent helps the user compose the e-mail message to be sent out. Most user agents provide a **template** on the screen to be filled in by the user.
- Some even have a **built-in editor** that can do spell checking, grammar checking, and other tasks expected from a sophisticated word processor.

Reading Messages

- When a user invokes a user agent, it first checks the mail in the incoming mailbox.
- Most user agents show a **one-line summary** of each received mail. Each e-mail contains the following fields.
 1. A number field.
 2. A flag field that shows the status of the mail such as new, already read but not replied to, or read and replied to.
 3. The size of the message.
 4. The sender.
 5. The optional subject field.

ELECTRONIC MAIL

Replying to Messages

- After reading a message, a user can use the user agent to reply to a message. A user agent usually allows the user to reply to the **original sender** or to **reply to all recipients** of the message. The reply message may contain the original message (for quick reference) and the new message.

Forwarding Messages

- Replying is defined as sending a message to the sender or recipients of the copy. **Forwarding is defined as sending the message to a third party.** A user agent allows the receiver to forward the message, with or without extra comments, to a third party.

Handling Mailboxes

- A user agent normally creates **two mailboxes**: an **inbox** and an **outbox**. Each box is a **file with a special format** that can be handled by the user agent.
- The inbox keeps all the received e-mails until they are deleted by the user. The outbox keeps all the sent e-mails until the user deletes them. Most user agents today are capable of creating customized mailboxes.

ELECTRONIC MAIL

User Agent Types

- There are two types of user agents: command-driven and GUI-based.
- **Command-Driven Command-driven**
- User agents belong to the early days of electronic mail. They are still present as the underlying **user agents in servers**. A command-driven user agent normally accepts a **one-character command** from the keyboard to perform its task. For example, a user can type the **character r, at the command prompt, to reply to the sender of the message, or type the character R to reply to the sender and all recipients**. **Some examples of command-driven user agents are mail, pine, and elm.**
- They contain graphical-user interface (GUI) components that allow the user to interact with the software by using both the **keyboard and the mouse**. They have graphical components such as **icons, menu bars, and windows** that make the services easy to access. **Some examples of GUI-based user agents are Eudora, Microsoft's Outlook, and Netscape.**

ELECTRONIC MAIL

Sending Mail

- To send mail, the user, through the UA, creates mail that looks very similar to postal mail. It has an envelope and a message.

Envelope: The envelope usually contains the sender and the receiver addresses.

Message

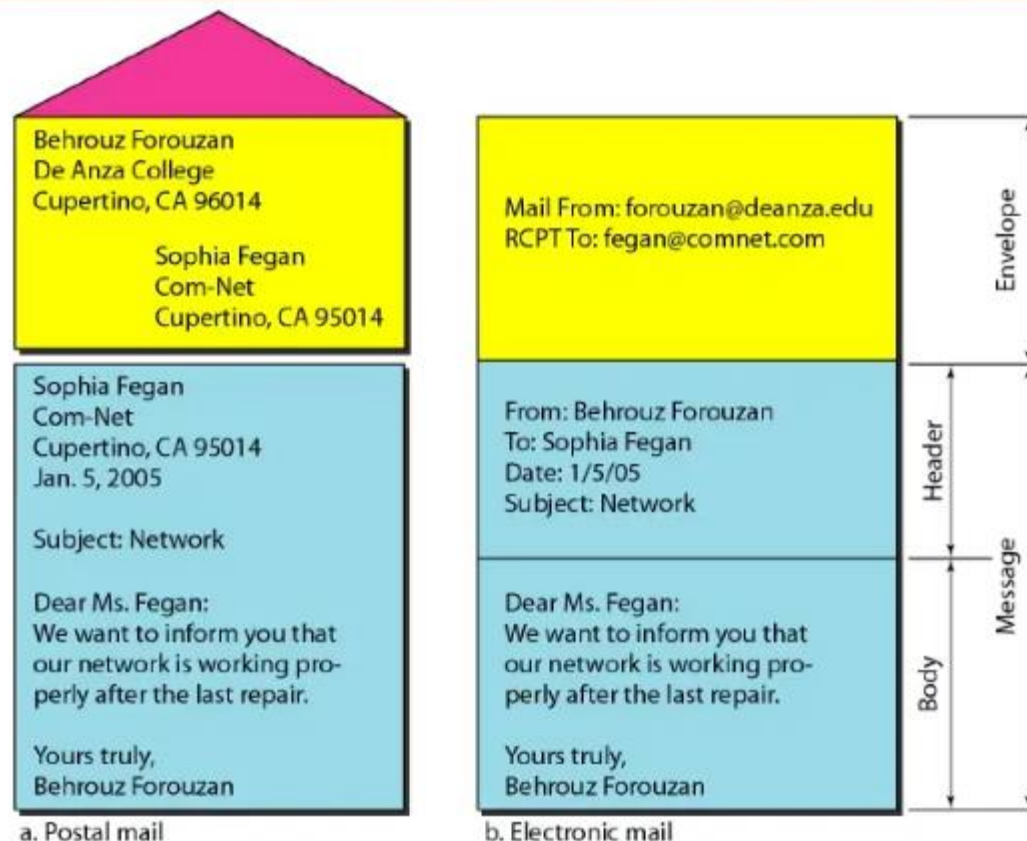
- The message contains the header and the body.
- The **header** of the message defines the sender, the receiver, the subject of the message, and some other information
- The **body** of the message contains the actual information to be read by the recipient.

ELECTRONIC MAIL

Sending Mail

- To send mail, the user, through the UA, creates mail that looks very similar to postal mail. It has an envelope and a message

Figure 26.12 *Format of an e-mail*



ELECTRONIC MAIL

Receiving Mail

- The user agent is triggered by the user (or a timer). If a user has mail, the UA informs the user with a **notice**.
- If the user is ready to read the **mail a list is displayed** in which each line contains a **summary** of the information about a particular message in the mailbox.
- The summary usually includes the **sender mail address, the subject, and the time the mail was sent or received**. The user can select any of the messages and display its contents on the screen.

ELECTRONIC MAIL

Addresses

- To deliver mail, a mail handling system must use an addressing system with unique addresses.
- In the Internet, the address consists of two parts: a local part and a domain name, separated by an @ sign

Figure 26.13 *E-mail address*



ELECTRONIC MAIL

Addresses

- **Local Part** The local part defines the name of a special file, called the user mailbox, where all the mail received for a user is stored for retrieval by the **message access agent**.
- **Domain Name** The second part of the address is the domain name. An organization usually selects one or more hosts to receive and send e-mail; the hosts are sometimes called mail servers or exchangers. The domain name assigned to each mail exchanger either comes from the DNS database or is a logical name

Figure 26.13 *E-mail address*

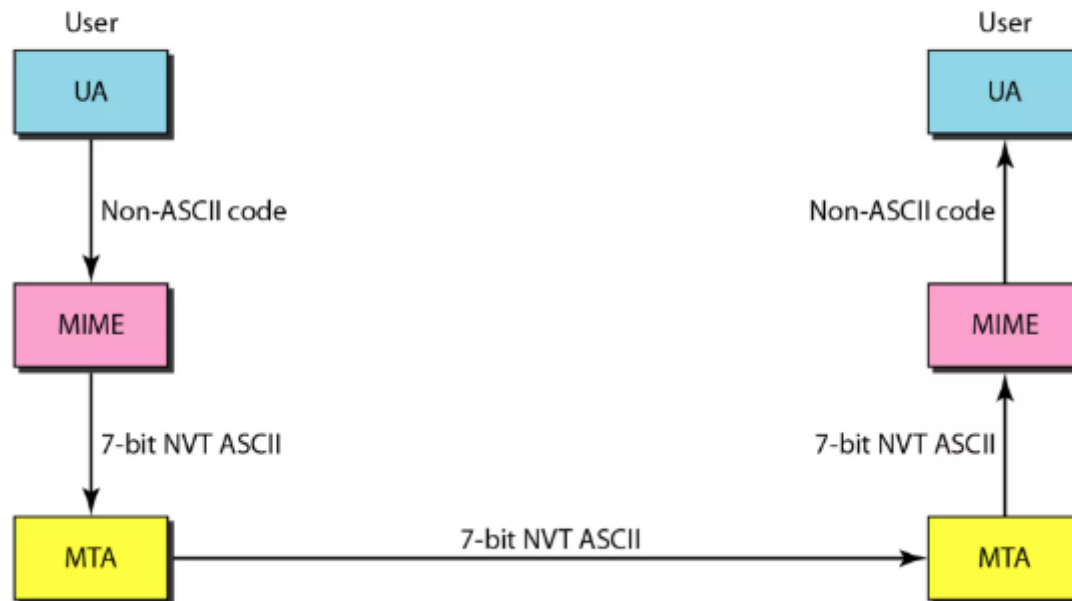


MIME

- **MIME- Multipurpose Internet Mail Extensions (MIME)**
- Electronic mail has a simple structure. It can send messages only in NVT 7-bit ASCII format.
- Electronic mail has some **limitations**.
 1. For example, it cannot be used for languages that are not supported by 7-bit ASCII characters (such as French, German, Hebrew, Russian, Chinese, and Japanese).
 2. It cannot be used to send binary files or video or audio data.
- **Multipurpose Internet Mail Extensions (MIME)** is a **supplementary protocol that allows non-ASCII data to be sent through e-mail**.
- MIME transforms non-ASCII data at the sender site to **NVT ASCII data** and delivers them to the client MTA to be sent through the Internet.
- The message at the receiving side is transformed back to the original data.
- MIME is a **set of software functions that transforms non-ASCII data** (stream of bits) to ASCII data and vice versa.

MIME

Figure 26.14 *MIME*

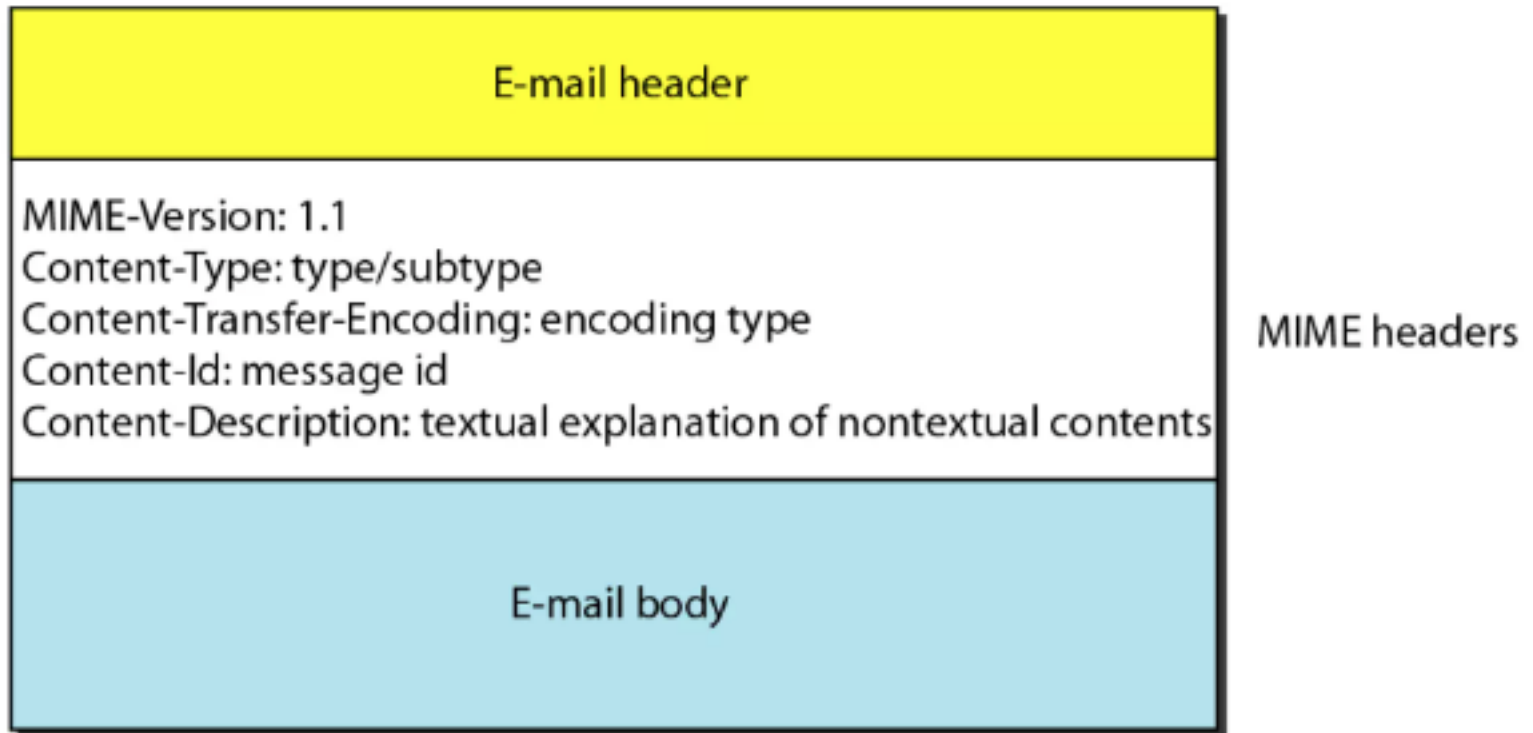


MIME

- **MIME**
- MIME defines five headers that can be added to the original e-mail header section to define the transformation parameters:
 1. MIME-Version
 2. Content-Type
 3. Content-Transfer-Encoding
 4. Content-Id
 5. Content-Description

MIME

Figure 26.15 *MIME header*



MIME

- **MIME-Version** This header defines the version of MIME used. The current version is **1.1**.
 - **Content-Type** This header defines the type of data used in the body of the message.
 - The content type and the content subtype are separated by a slash. Depending on the subtype, the header may contain other parameters.
- Content-Type:<type/subtype:parameters>.**
- MIME allows seven different types of data

Table 26.5 *Data types and subtypes in MIME*

Type	Subtype	Description
Text	Plain	Unformatted
	HTML	HTML format (see Chapter 27)
Multipart	Mixed	Body contains ordered parts of different data types
	Parallel	Same as above, but no order
	Digest	Similar to mixed subtypes, but the default is message/RFC822
	Alternative	Parts are different versions of the same message
Message	RFC822	Body is an encapsulated message
	Partial	Body is a fragment of a bigger message
	External-Body	Body is a reference to another message
Image	JPEG	Image is in JPEG format
	GIF	Image is in GIF format
Video	MPEG	Video is in MPEG format
Audio	Basic	Single-channel encoding of voice at 8 kHz
Application	PostScript	Adobe PostScript
	Octet-stream	General binary data (8-bit bytes)

MIME

- **Content-Transfer-Encoding** This header defines the method used to encode the messages into 0s and 1s for transport:
Content-Transfer-Encoding:<type>
- The five types of encoding methods are listed in Table 26.6.

Table 26.6 *Content-transfer-encoding*

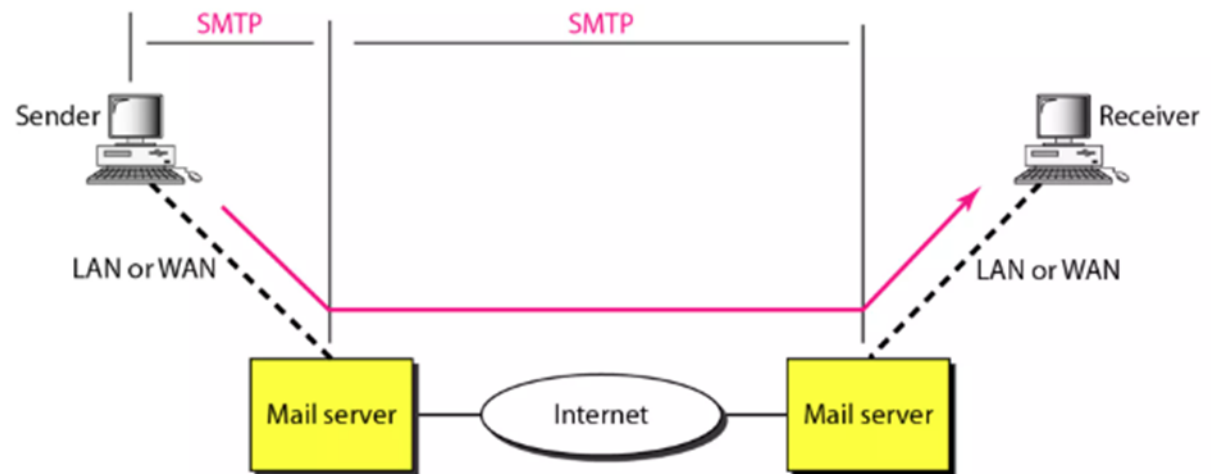
<i>Type</i>	<i>Description</i>
7-bit	NVT ASCII characters and short lines
8-bit	Non-ASCII characters and short lines
Binary	Non-ASCII characters with unlimited-length lines
Base-64	6-bit blocks of data encoded into 8-bit ASCII characters
Quoted-printable	Non-ASCII characters encoded as an equals sign followed by an ASCII code

MIME

- **Content-Id** This header uniquely identifies the whole message in a multiple-message environment.
 - **Content-Id:** id=<content-id>
- **Content-Description** This header defines whether the body is image, audio, or video.
 - **Content-Description:** <description>

SMTP: Simple Mail Transfer Protocol

- **Message Transfer Agent: SMTP**
- The actual mail transfer is done through message transfer agents.
- To send mail, a system must have the client MTA, and to receive mail, a system must have a server MTA.
- The formal protocol that defines the MTA client and server in the Internet is called the **Simple Mail Transfer Protocol (SMTP)**.
- Two pairs of MTA client/server programs are used in the most common situation (fourth scenario). Figure 26.16 shows the range of the SMTP protocol in this scenario. **Figure 26.16** *SMTP range*



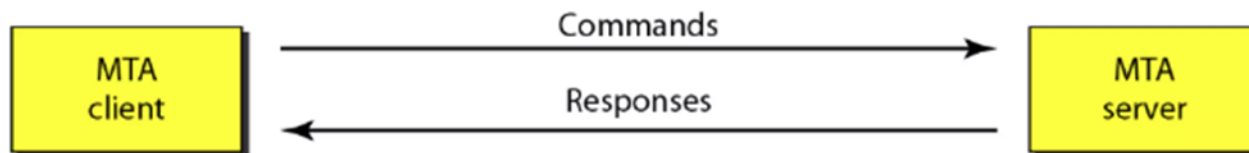
SMTP: Simple Mail Transfer Protocol

- SMTP is used two times, between the sender and the sender's mail server and between the two mail servers.
- **Another protocol is needed between the mail server and the receiver.**
- SMTP simply defines how commands and responses must be sent back and forth.
- Each network is free to choose a software package for implementation.

Commands and Responses

- SMTP uses commands and responses to transfer messages between an MTA client and an MTA server

Figure 26.17 *Commands and responses*



SMTP: Simple Mail Transfer Protocol

- Each command or reply is terminated by a two-character (carriage return and line feed) end-of-line token.
- **Commands**
- Commands are sent from the client to the server.
- The format of a command consists of a keyword followed by zero or more arguments.
- SMTP defines 14 commands. The first five are mandatory; every implementation must support these five commands. The next three are often used and highly recommended. The last six are seldom used.

Figure 26.18 *Command format*



The diagram shows a rectangular box with a double border. Inside the box, the text "Keyword: argument(s)" is displayed. The word "Keyword:" is in pink, and "argument(s)" is in black.

Keyword: argument(s)

SMTP: Simple Mail Transfer Protocol

Table 26.7 *Commands*

<i>Keyword</i>	<i>Argument(s)</i>
HELO	Sender's host name
MAIL FROM	Sender of the message
RCPT TO	Intended recipient of the message
DATA	Body of the mail
QUIT	
RSET	
VERFY	Name of recipient to be verified
NOOP	
TURN	
EXPN	Mailing list to be expanded
HELP	Command name
SEND FROM	Intended recipient of the message
SMOL FROM	Intended recipient of the message
SMAL FROM	Intended recipient of the message

SMTP: Simple Mail Transfer Protocol

Responses Responses are sent from the server to the client. A response is a three digit code that may be followed by additional textual information. Responses are divided into four categories. The leftmost digit of the code (2, 3, 4, and 5) defines the category.

Table 26.8 *Responses*

<i>Code</i>	<i>Description</i>
Positive Completion Reply	
211	System status or help reply
214	Help message
220	Service ready
221	Service closing transmission channel
250	Request command completed
251	User not local; the message will be forwarded
Positive Intermediate Reply	
354	Start mail input
Transient Negative Completion Reply	
421	Service not available
450	Mailbox not available
451	Command aborted: local error
452	Command aborted: insufficient storage

Table 26.8 *Responses (continued)*

<i>Code</i>	<i>Description</i>
Permanent Negative Completion Reply	
500	Syntax error; unrecognized command
501	Syntax error in parameters or arguments
502	Command not implemented
503	Bad sequence of commands
504	Command temporarily not implemented
550	Command is not executed; mailbox unavailable
551	User not local
552	Requested action aborted; exceeded storage location
553	Requested action not taken; mailbox name not allowed
554	Transaction failed

SMTP: Simple Mail Transfer Protocol

Mail Transfer Phases

The process of transferring a mail message occurs in **three phases: connection establishment, mail transfer, and connection termination.**



Example 26.3



Let us see how we can directly use SMTP to send an e-mail and simulate the commands and responses we described in this section. We use TELNET to log into port 25 (the well-known port for SMTP). We then use the commands directly to send an e-mail. In this example, forouzanb@adelphia.net is sending an e-mail to himself. The first few lines show TELNET trying to connect to the Adelphia mail server. After connection, we can type the SMTP commands and then receive the responses, as shown on the next slide. Note that we have added, for clarification, some comment lines, designated by the “=” signs. These lines are not part of the e-mail procedure.

SMTP: Simple Mail Transfer Protocol

\$ telnet mail.adelphia.net 25

Trying 68.168.78.100 . . .

Connected to mail.adelphia.net (68.168.78.100).

===== Connection Establishment =====

220 mta13.adelphia.net SMTP server ready Fri, 6 Aug 2004 . . .

HELO mail.adelphia.net

250 mta13.adelphia.net

SMTP: Simple Mail Transfer Protocol

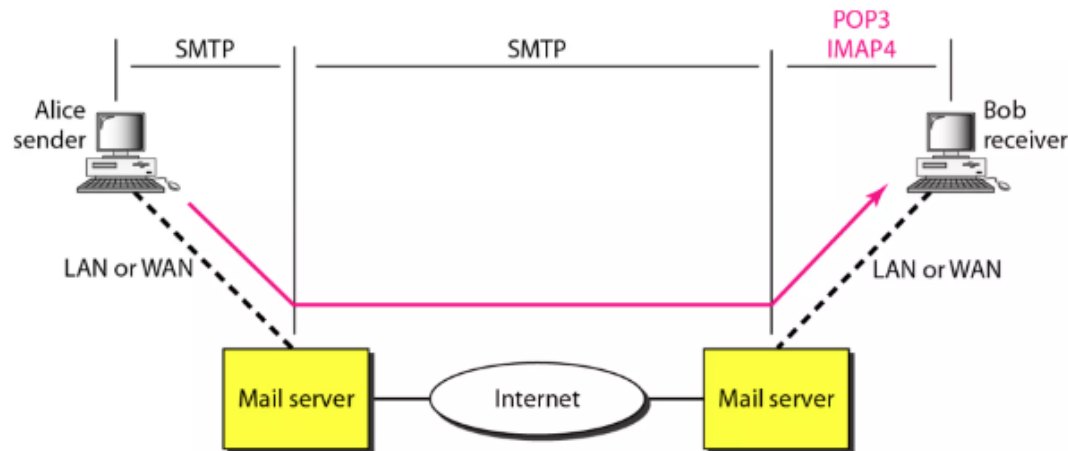
```
===== Mail Transfer =====  
MAIL FROM: forouzanb@adelphia.net  
    250 Sender <forouzanb@adelphia.net> Ok  
RCPT TO: forouzanb@adelphia.net  
    250 Recipient <forouzanb@adelphia.net> Ok  
DATA  
    354 Ok Send data ending with <CRLF>.<CRLF>  
From: Forouzan  
TO: Forouzan  
  
This is a test message  
to show SMTP in action.  
.
```

```
===== Connection Termination =====  
    250 Message received: adelphia.net@mail.adelphia.net  
QUIT  
    221 mta13.adelphia.net SMTP server closing connection  
Connection closed by foreign host.
```

Message Access Agent: POP and IMAP

- **Message Access Agent: POP and IMAP**
- The first and the second stages of mail delivery use SMTP.
- SMTP is not involved in the third stage because SMTP is a **push protocol**; it pushes the message from the client to the server.
- The third stage needs a **pull protocol**; the client must pull messages from the server. The direction of the bulk data is from the server to the client. The third stage uses a **message access agent**.
- Currently two message access protocols are available: Post Office Protocol, version 3 (**POP3**) and Internet Mail Access Protocol, version 4 (**IMAP4**).

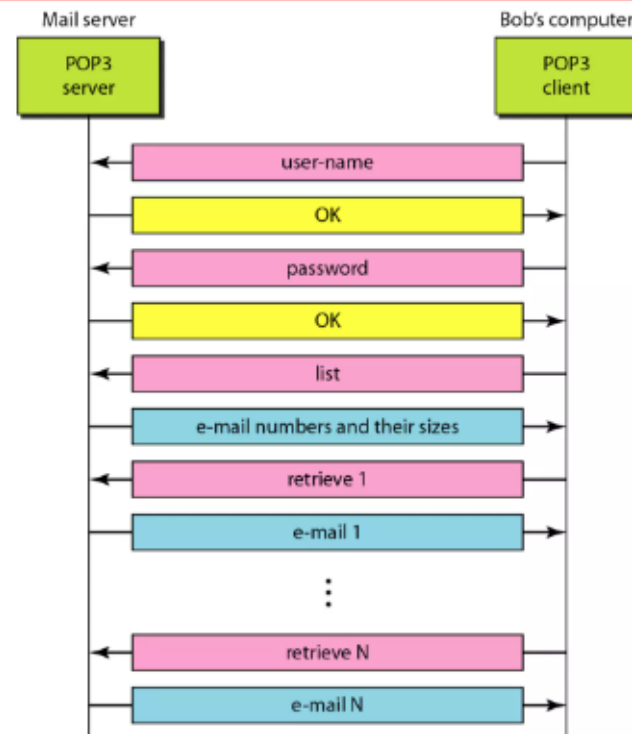
Figure 26.19 *POP3 and IMAP4*



Message Access Agent: POP and IMAP

- **Post Office Protocol, version 3 (POP3)**
- It is simple and limited in functionality. The **client POP3** software is installed on the recipient computer; the **server POP3** software is installed on the mail server.
- Mail access starts with the client when the user needs to **download e-mail** from the mailbox on the mail server. The client opens a connection to the server on TCP port **110**. It then sends its user name and password to access the mailbox. The user can then list and retrieve the mail messages, one by one.

Figure 26.20 *The exchange of commands and responses in POP3*



Message Access Agent: POP and IMAP

Post Office Protocol, version 3 (POP3)

- POP3 has two modes: the delete mode and the keep mode.
- In the delete mode, the mail is deleted from the mailbox after each retrieval. In the keep mode, the mail remains in the mailbox after retrieval. The delete mode is normally used when the user is working at her permanent computer and can save and organize the received mail after reading or replying.
- The keep mode is normally used when the user accesses her mail away from her primary computer (e.g., a laptop). The mail is read but kept in the system for later retrieval and organizing.

Message Access Agent: POP and IMAP

Internet Mail Access Protocol, version 4 (IMAP4).

- IMAP4 is similar to POP3, but it has more features; IMAP4 is more powerful and more complex.
- POP3 is deficient in several ways. It **does not allow the user to organize her mail** on the server; the user cannot have different folders on the server. (Of course, the user can create folders on her own computer.) In addition, POP3 does not allow the **user to partially check the contents** of the mail before downloading.
- **IMAP4 provides the following extra functions:**
 - A user can **check the e-mail header** prior to downloading.
 - A user can **search** the contents of the e-mail for a specific string of characters prior to downloading.
 - A user can **partially download e-mail**. This is especially useful if bandwidth is limited and the e-mail contains multimedia with high bandwidth requirements.
 - A user can **create, delete, or rename mailboxes** on the mail server.
 - A user can **create a hierarchy of mailboxes** in a folder for e-mail storage.

Message Access Agent: POP and IMAP

Web-Based Mail

- E-mail is such a common application that some websites today provide this service to anyone who accesses the site. The idea is very simple. Mail transfer from Alice's browser to her mail server is done through HTTP.
- The transfer of the message from the sending mail server to the receiving mail server is still through SMTP.
- Finally, the message from the receiving server (the Web server) to Bob's browser is done through HTTP. In last phase instead of POP3 or IMAP4, HTTP is normally used.

FILE TRANSFER

- Transferring files from one computer to another is one of the most common tasks expected from a networking or internetworking environment.
- The **greatest volume of data exchange** in the Internet today is due to file transfer.
- One popular protocol involved in transferring files: File Transfer Protocol (FTP).
- File Transfer Protocol (FTP) is the standard mechanism provided by **TCP/IP** for copying a file from one host to another.
- Transferring files from one system to another seems simple and straightforward, **some problems must be dealt**.
- For example,
 - Two systems may use **different file name conventions**.
 - Two systems may have **different ways to represent text and data**.
 - Two systems may have **different directory structures**.All these **problems have been solved by FTP** in a very simple and elegant approach.

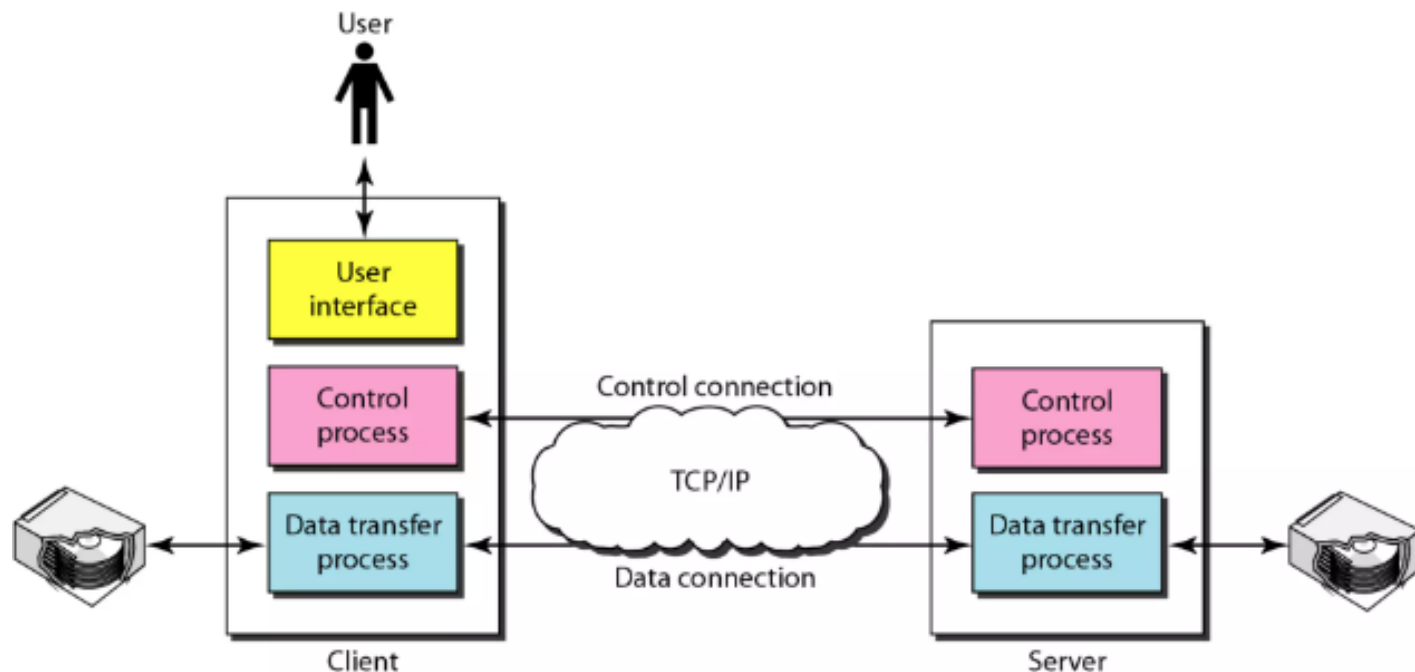
FILE TRANSFER

- FTP differs from other client/server applications in that it **establishes two connections between the hosts**.
- One connection is used for **data transfer**, the other for **control information (commands and responses)**.
- Separation of commands and data transfer makes FTP more efficient.
- The control connection uses very **simple rules of communication**.
- Need to transfer only a **line of command or a line of response** at a time.
- For TCP, both connections are treated the same.
- FTP uses two well-known TCP ports: **Port 21 is used for the control connection, and port 20 is used for the data connection**.

FILE TRANSFER

- **The client has three components:** user interface, client control process, and the client data transfer process.
- **The server has two components:** the server control process and the server data transfer process.
- The control connection is made between the control processes. The data connection is made between the data transfer processes.

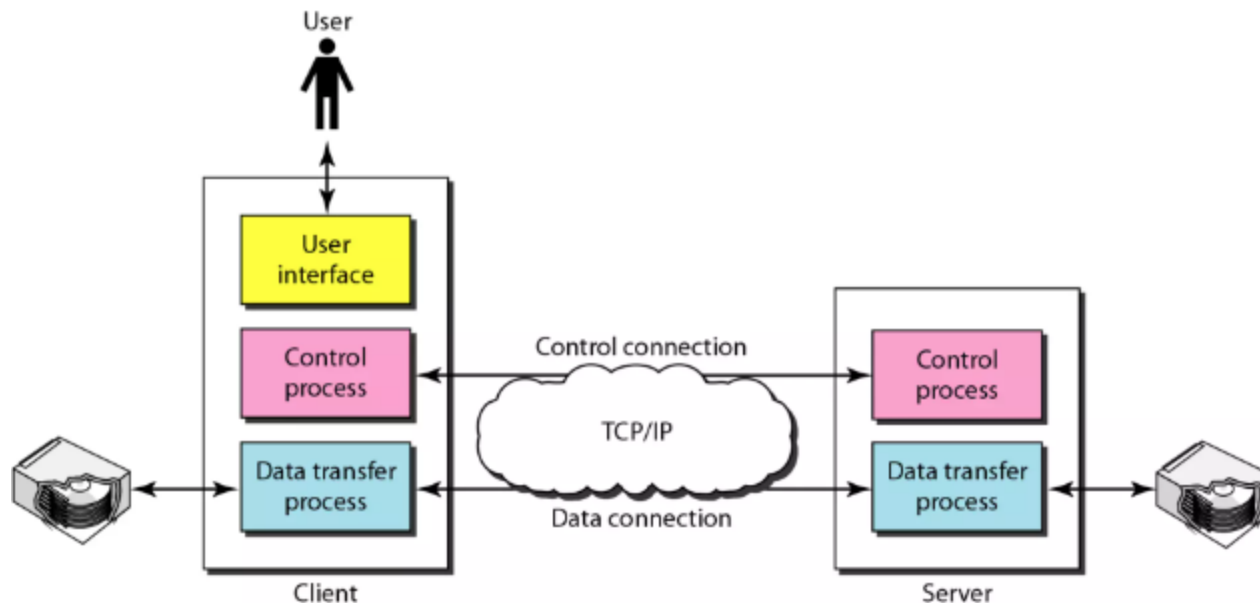
Figure 26.21 *FTP*



FILE TRANSFER

- The control connection remains connected during the entire interactive FTP session.
- The data connection is opened and then closed for each file transferred.
- It opens each time commands that involve transferring files are used, and it closes when the file is transferred.
- In other words, when a user starts an FTP session, the control connection opens.
- While the control connection is open, the data connection can be opened and closed multiple times if several files are transferred.

Figure 26.21 *FTP*

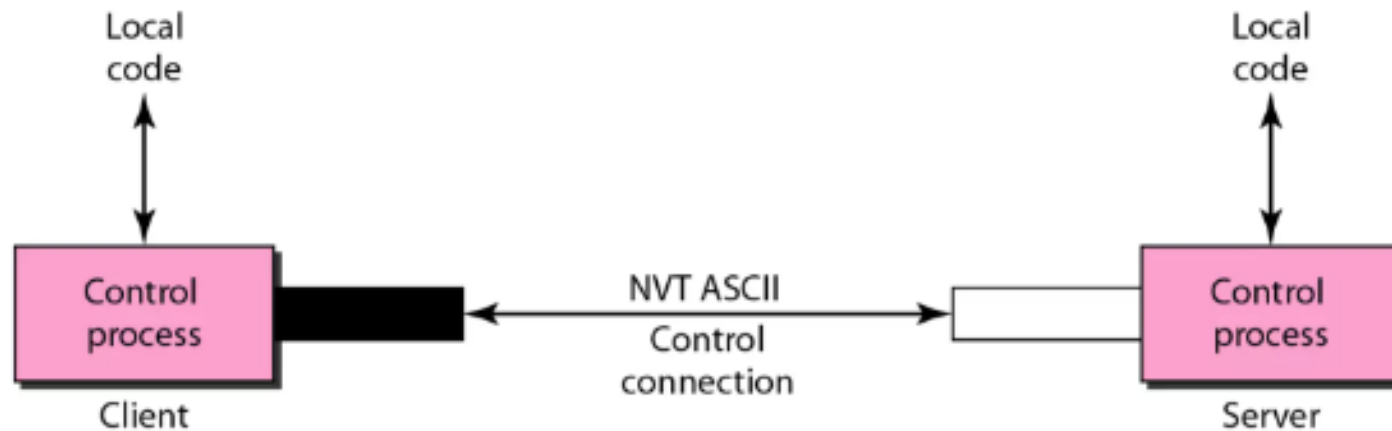


FILE TRANSFER

- **Communication over Control Connection**
- FTP uses the same approach as SMTP to communicate across the control connection.
- It uses the 7-bit ASCII character set. Communication is achieved through commands and responses.
- This simple method is adequate for the control connection because we send one command (or response) at a time. Each command or response is only one short line, so we need not worry about file format or file structure.
- Each line is terminated with a two-character (carriage return and line feed) end-of-line token.

FILE TRANSFER

Figure 26.22 *Using the control connection*



FILE TRANSFER

- **Communication over Data Connection**
 - Transfer files through the data connection.
 - File transfer occurs over the data connection under the control of the commands sent over the control connection.
 - File transfer in FTP means one of three things:
 1. A file is to be copied from the server to the client. This is called retrieving a file. It is done under the supervision of the **RETR** command.
 2. A file is to be copied from the client to the server. This is called storing a file. It is done under the supervision of the **STOR** command.
 3. A list of directory or file names is to be sent from the server to the client. This is done under the supervision of the **LIST** command. Note that FTP treats a list of directory or file names as a file. It is sent over the data connection.
- The client must define the type of file to be transferred, the **structure of the data, and the transmission mode**. Before sending the file through the data connection, prepare for transmission through the control connection. The heterogeneity problem is resolved by defining three attributes of communication: **file type, data structure, and transmission mode**

FILE TRANSFER

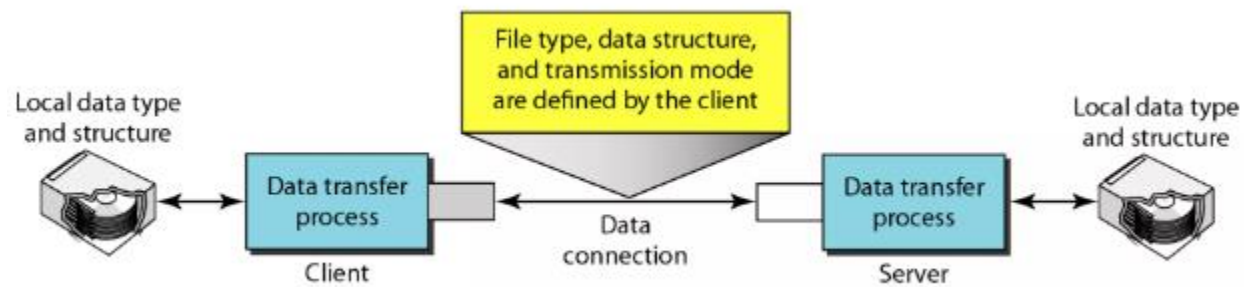
- **Communication over Data Connection**
- **File Type:** FTP can transfer one of the following file types across the data connection: an ASCII file, EBCDIC file, or image file. The ASCII file is the default format for transferring text files. Each character is encoded using 7-bit ASCII. The sender transforms the file from its own representation into ASCII characters, and the receiver transforms the ASCII characters to its own representation.
- **Data Structure :** FTP can transfer a file across the data connection by using one of the following interpretations about the structure of the data: file structure, record structure, and page structure. In the file structure format, the file is a continuous stream of bytes. In the record structure, the file is divided into records. This can be used only with text files. In the page structure, the file is divided into pages, with each page having a page number and a page header. The pages can be stored and accessed randomly or sequentially.

FILE TRANSFER

- **Transmission Mode:** FTP can transfer a file across the data connection by using one of the following three transmission modes: stream mode, block mode, and compressed mode.
- The **stream mode** is the default mode. Data are delivered from FTP to TCP as a continuous stream of bytes. TCP is responsible for chopping data into segments of appropriate size. If the data are simply a stream of bytes (file structure), no end-of-file is needed. End-of-file in this case is the closing of the data connection by the sender.
- In **block mode**, data can be delivered from FTP to TCP in blocks. In this case, each block is preceded by a 3-byte header. The first byte is called the block descriptor; the next 2 bytes define the size of the block in bytes.
- In the **compressed mode**, if the file is big, the data can be compressed. The compression method normally used is run-length encoding.

FILE TRANSFER

Figure 26.23 *Using the data connection*



FILE TRANSFER

```
$ ftp voyager.deanza.fhda.edu
Connected to voyager.deanza.fhda.edu.
220 (vsFTPd 1.2.1)
530 Please login with USER and PASS.
Name (voyager.deanza.fhda.edu:forouzan): forouzan
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls reports
227 Entering Passive Mode (153,18,17,11,238,169)
150 Here comes the directory listing.
drwxr-xr-x  2 3027      411          4096 Sep 24  2002 business
drwxr-xr-x  2 3027      411          4096 Sep 24  2002 personal
drwxr-xr-x  2 3027      411          4096 Sep 24  2002 school
226 Directory send OK.
ftp> quit
221 Goodbye.
```

FILE TRANSFER

We show an example of anonymous FTP. We assume that some public data are available at internic.net.

```
$ ftp internic.net  
Connected to internic.net  
220 Server ready  
Name: anonymous  
331 Guest login OK, send "guest" as password  
Password: guest
```

continued on next slide

FILE TRANSFER



Example 26.5 (continued)

5

```
ftp > pwd
257 '/' is current directory
ftp > ls
200 OK
150 Opening ASCII mode
bin
...
...
...
ftp > close
221 Goodbye
ftp > quit
```